Memory Footprints from Resident Memory and Allocator Statistics: A Footprint Model at the Cost of a Native Run

Nafis Mustakin
University of California, Riverside

October 2026

Abstract

A memory-aware scheduler needs to know how much memory a job uses over its run. MemPrint, a Pin-based footprint profiler, measures the pages a program touches, but its windowed mode costs 12.0× a native run at the median of 22 programs, 18.7× to 93.0× on programs that allocate and free memory often, and up to 67.4× on a language model engine. The resident set size (RSS) of a native run costs nothing extra, but it overestimates allocator-heavy programs by a median of 16% over the whole run, and miniVite by 102%, because the allocator keeps freed memory resident and RSS counts shared libraries. We propose a footprint model that samples, from one thread inside the process every 5 ms, the anonymous RSS, the proportional set size, the RSS of mapped data files, and the free bytes the allocator reports (glibc, jemalloc or TCMalloc). Its data estimate, anonymous RSS minus free bytes plus mapped data files and the runtime's pages, has a median whole-run error of −4% on the 7 allocator-heavy programs of a 23-program corpus (−7% on miniVite), against −5% for MemPrint, at a median of 0.99× the run time of a plain RSS run. Transferred from smaller inputs to a held-out larger one, it has a median error of −4%, against +20% for RSS and −6% for MemPrint. In a cluster scheduling simulator that reserves memory by each job's estimated curve, it gives the lowest mean response time of all estimates that profile the job's own input (433 minutes at a load of 0.8, against 515 for the RSS curve and 519 for MemPrint). Under TCMalloc its median error is −4%. Under jemalloc it is −1% with transparent huge pages disabled and +35% with them enabled, because the kernel backs memory the program touches in part with whole 2 MiB pages. On llama2.c and llama.cpp, the data-file term brings the model from between −39% and −92% to between −28% and +15% of the truth. The same library also measures which memory is cold, from the pages' accessed bits per call site and 2 MiB chunk, at 1.03× to 1.21× a native run. On a remote NUMA node that stands in for a memory pool, a plan built from one profile run, even on a smaller input, keeps a quarter of the memory-time in the pool at 1.000× to 1.008× the run time on GAP and CPython, where placing the hottest quarter costs up to 1.135×, and moving memory between nodes during the run costs more than the plan. A latency model, checked at two pool latencies, predicts that the plan costs at most 1.07× on a pool at four times the local latency, where all memory in the pool costs up to 1.94×; in a simulated cluster, the cold plan then runs 4% more jobs per hour than a random one.

1 Introduction

A scheduler that packs jobs onto nodes by their memory use needs an estimate of each job's footprint over time, because a job's peak lasts only part of its run [2, 25, 12]. A reservation that follows the footprint over time lets a node hold more jobs than a reservation of the peak, but the estimate must not fall below what the job uses, or the job runs out of memory.

MemPrint measures a program's footprint with Pin [14]. It records the bytes and pages each program touches and the time at which each block is freed. Its windowed mode watches 5% of the run and costs 12.0× a native run at the median on 22 programs of our corpus. However, on the programs where a footprint estimate helps the most, those that allocate and free memory throughout their run, the cost rises to 18.7× to 93.0×, because Pin instruments every memory access and every allocator call.

The resident set size of a native run is the obvious cheap alternative. On array programs, RSS is the footprint, because these programs allocate their arrays once and touch them. On allocator-heavy programs, RSS overestimates the footprint over the run by a median of 16%, and by 102% on miniVite [7], because the kernel counts memory that the program no longer uses.

In this work, we make the following contributions:

- We identify the parts of RSS that are not part of the footprint, freed memory that the allocator keeps resident and the pages of shared libraries, and we measure their size per program (Section 5.1).
- We propose a footprint model that combines RSS, PSS, the RSS of mapped data files and the allocator's free bytes, sampled every 5 ms from a thread inside the process through an `LD_PRELOAD` library (Section 3).
- We show on 23 programs that the model matches MemPrint's accuracy on the job's own input and on larger inputs predicted from smaller ones, at a median of 0.99× the cost of a plain RSS run, and that sampling every 1 ms at the same cost corrects the transfer from runs of a few milliseconds (Section 5 and Section 6).
- We show in a cluster scheduling simulator that the model's curve gives the lowest response time of all estimates of the job's own input (Section 7).
- We evaluate the model under jemalloc and TCMalloc and on two language model engines, and we show that transparent huge pages are the remaining source of disagreement with a 4 KiB pages-touched truth, which no interface without privileges can resolve inside a huge page (Section 8 and Section 9).
- We test four other ways to correct RSS, each alone, and show why each is less accurate (Section 11).
- We use the model at run time in libmpr, our allocation-resizing runtime, to return glibc's free memory to the kernel from `free`, which lowers the time-averaged RSS of the allocator-heavy programs by 4.4% at a median run time of 1.01×, half of what returning every free byte would give (Section 12).
- We extend the sampling library to measure cold memory per call site from the pages' accessed bits, without privileges, and we show on an emulated memory pool that a placement plan from one profile run, carried to a larger input, keeps a quarter of a job's memory-time in the pool at 0 to 17% of the cost of placing all of it there, that moving memory during the run does not pay on our programs, and that libmpr applies the plan together with its reservations (Section 13).
- We show that the model needs no earlier run of a job, that it is within a median of 2.0% of the truth on the blind programs of a companion study [16], that borrowing the growth of similar programs does not beat proportional growth, that code embeddings do not capture

growth either, and that the companion study's interpreter predicts array programs with no run at all (Section 10).

- We compare the costs and effects of the approaches to the gap between RSS and the footprint, from estimating it to lowering it and reserving less than the request (Section 14).

2 Background: what RSS counts

The RSS of a process is the number of its pages in memory, which the kernel counts in three parts that `/proc/pid/status` reports as `RssAnon`, `RssFile` and `RssShmem`:

$$\mathrm{RSS} = \mathrm{RSS}_{\mathrm{anon}} + \mathrm{RSS}_{\mathrm{file}} + \mathrm{RSS}_{\mathrm{shmem}}. \tag{1}$$

An *anonymous* page is a page that no file backs. The heap, the stacks and the memory that `mmap` returns with `MAP_ANONYMOUS` are anonymous, which covers every block that `malloc` serves. The kernel fills an anonymous page with zeros when the program first touches it, and if the kernel needs the page for something else, it can only move the contents to swap. The *anonymous RSS*, $\mathrm{RSS}_{\mathrm{anon}}$, counts the anonymous pages of the process that are in memory. An anonymous page enters RSS when the program first touches it and leaves when the program or its allocator unmaps it or releases it with `madvise`. A *file-backed* page holds part of a file, such as a page of a shared library's code or of a mapped data file. It enters RSS when any process reads it and stays as long as it is mapped and the kernel does not reclaim it, which the kernel can do at any time by dropping the page and reading it from the file again later. $\mathrm{RSS}_{\mathrm{shmem}}$ counts shared memory, such as `/dev/shm` files and shared anonymous mappings, which none of our programs use. The *footprint* we estimate is the set of pages the program has touched and not yet freed, plus the pages the runtime needs before the program starts. RSS differs from the footprint in five ways.

Allocator retention. The C library does not return every freed block to the kernel. glibc's `malloc` serves blocks below a threshold (128 KiB, raised dynamically up to 32 MiB) from its arenas, keeps freed blocks in free lists for reuse, and releases memory only from the top of the heap when the free space there exceeds the trim threshold [6]. jemalloc and TCMalloc also release freed memory late or in batches [5, 8]. A program that frees and allocates memory throughout its run therefore holds freed pages in RSS. Each of these allocators reports the bytes it holds free through its own interface.

Shared libraries. RSS counts every resident page of every mapped file, including the code of the C library, the C++ runtime and, for MPI programs, the MPI library. These pages do not depend on the input, and other processes on the node map the same pages. The *proportional set size* (PSS) of a process charges the process for each of its resident pages in proportion to how many processes map that page. For a process whose resident pages form the set R, with a page size of s bytes,

$$\mathrm{PSS} = \sum_{p \in R} \frac{s}{n_p}, \tag{2}$$

where n_p is the number of processes that map page p. The kernel computes this sum over the page tables of the process when a program reads `/proc/pid/smaps` or `smaps_rollup`, which is why PSS costs more to read than the counters of `/proc/pid/status`. A page that only this process maps counts in full ($n_p = 1$), so the anonymous pages of a process that does not share memory with its parent or children count in full, and PSS is $\mathrm{RSS}_{\mathrm{anon}}$ plus each resident file-backed page divided by the number of processes that map it. The PSS of all processes on a node sums to the memory of the resident pages they map, whereas their RSS counts each shared page once per process.

Mapped data files. A program that maps its input, such as a language model engine that maps its weights, holds that input in file-backed pages. These pages are part of the footprint, unlike the pages of libraries.

Language runtimes with their own allocators. CPython's object allocator takes 256 KiB arenas from the system and returns an arena only when all its objects are freed. GCC's garbage-collected heap also maps its own pages. The C library's statistics do not cover these allocators.

Page size. RSS counts whole pages. On our corpus, the 4 KiB pages a program touches exceed the bytes it touches by 1% at the median, so we use the pages as the truth. With transparent huge pages [13], the kernel can back a 2 MiB aligned range of anonymous memory with one huge page at the first touch, and RSS then counts 2 MiB even if the program touches only part of the range [10, 20]. Our machine enables transparent huge pages for all anonymous memory (the `always` setting).

3 The footprint model

The model estimates a footprint curve from five quantities sampled at the same time inside the process. Let $A(t)$ be the anonymous RSS, $P(t)$ the PSS, $M(t)$ the RSS of mapped data files, $F(t)$ the bytes the allocator holds free, and B the runtime's own pages, the smallest RSS of an earlier run of the program, which is measured once per program. The model gives two estimates.

Estimate 1, the data footprint, is

$$\hat{f}_1(t) = \max\big(A(t) - F(t),\, 0\big) + M(t) + B. \tag{3}$$

It counts the anonymous pages the allocator has not freed and the pages of mapped data files, and it leaves out the pages of libraries. It estimates the memory that grows with the input, which is what a scheduler predicts and what our truth measures.

Estimate 2, the charged footprint, is

$$\hat{f}_2(t) = \max\big(P(t) - F(t),\, 0\big). \tag{4}$$

PSS is the anonymous RSS plus each file-backed page divided by the number of processes that map it, so the estimate charges the job for its share of shared libraries and data files and for all of a library that only it maps. It estimates the memory the job adds to a node.

Both estimates subtract free bytes from resident memory rather than count live allocations. A counter of live `malloc` blocks misses every allocation that does not go through `malloc`, so an estimate built from it falls below the truth whenever a program uses its own allocator. Subtracting free bytes errs the other way. Memory that the allocator does not report stays in RSS, so a program whose allocator the model cannot see gets the RSS estimate, which is too high but safe.

Free bytes. The library looks up the allocator's interface when it starts. With jemalloc (`mallctl`), the free bytes are `stats.resident` minus `stats.allocated` minus `stats.metadata`, after a refresh of the statistics through `epoch`. With TCMalloc (`MallocExtension_GetNumericProperty`), they are `generic.heap_size` minus `tcmalloc.pageheap_unmapped_bytes` minus `generic.current_allocated_bytes`. Otherwise they are glibc's `mallinfo` field `fordblks`, the free bytes of all its arenas. The machine runs glibc 2.28, which has no `mallinfo2`, so glibc's free bytes are a 32-bit field that wraps above 4 GiB. A newer glibc reports them in 64 bits.

Mapped data files. A data file is a file mapping that is not the executable, a shared object (a path that contains `.so`), or the C library's locale and character set data. $M(t)$ is the sum of the RSS of these mappings.

Implementation. A shared library of 162 lines of C, loaded with `LD_PRELOAD`, starts one thread in its constructor. Every 5 ms the thread reads the process's memory counters, queries the allocator, and writes one line to a buffered file. It has two modes. The first reads `/proc/self/smaps_rollup`, which gives A, P and RSS but not M. The second reads `/proc/self/smaps`, one record per mapping, which also gives M. On the 23 corpus programs, M is at most 0.07 MiB, so we evaluate the corpus with the first mode and the language model engines with the second. The thread does not stop the program's threads, and it takes the allocator's locks only for the duration of its query. The library removes itself from `LD_PRELOAD` so that child processes are not sampled, and a program made of several processes would need one output file per process.

4 Methodology

Machine. All runs use a two-socket Intel Xeon E5-2620 v4 (16 cores, 32 threads) with 62 GiB of memory, Linux 4.18 with transparent huge pages enabled, and glibc 2.28. Runs whose time we report execute one at a time on an otherwise idle machine.

Corpus. We use the 23 programs of the corpus for which a pages-touched truth exists, each at its largest input: PolyBench (2mm, atax, gemm, jacobi-2d), the NAS parallel benchmarks (cg, is, mg), the GAP benchmark suite (bfs, cc, pr, sssp) [3], the proxy applications HPCCG, Kripke, LULESH, miniFE, miniVite and Quicksilver, darknet AlexNet inference, GCC's `cc1` compiling SQLite, and Lua [9], CPython, Perl and SQLite running table, JSON, hash and SQL workloads. We call 7 programs *allocator-heavy*, because they allocate and free memory throughout their run: miniVite, CPython, Lua, SQLite, Perl, Quicksilver and `cc1`. We chose the set before these experiments, from the earlier observation that RSS and MemPrint disagree on these programs and agree on the others. The runs last 0.1 to 12 s and peak at 27 to 273 MiB of RSS. Every program except darknet also has one or two smaller inputs. Separately, we use llama2.c with three model sizes (15M, 42M and 110M parameters) and llama.cpp with a 135M-parameter model generating 32 and 128 tokens.

Truth. The truth is the number of 4 KiB pages the program touches and has not freed, which Pin measures in MemPrint's splitter mode with page counting and free tracking, plus the runtime's pages B. Pin's time axis is its count of memory references. We place the truth on the wall clock of a native run through hardware counts of the loads and stores that the native run executes in every 5 ms interval. The truth at a share f of Pin's references is placed at the time at which the native run had executed a share f of its references. We divide each run into 100 equal steps of progress and take the largest value of each curve in each step. The estimates are placed on the same 100 steps by the wall clock of their own run. The truths under jemalloc and TCMalloc come from Pin runs with the same allocator, and they are placed on the wall clock by the counts of the native run with glibc.

Metrics. We report three errors of an estimate e against the truth t over the 100 steps. The signed whole-run error is $(\sum e - \sum t)/\sum t$, which is positive when the estimate reserves more memory over the run than the program uses. The absolute whole-run error is $\sum |e - t|/\sum t$. The peak error is $\max e/\max t - 1$. Each program runs 3 times, and we report the median of each error over the runs. The overhead of an estimate is the median wall time of its run over the median wall time of a run under the plain RSS sampler of the corpus (`rss_sampler.py`, an external process that reads `smaps_rollup` every 5 ms and takes 40 ms to start).

Baselines. *RSS* is the RSS curve of the plain sampler's run. *MemPrint* is the windowed mode that watches 5% of the run (W5), placed on the native wall clock by the same reference counts as the truth, plus B. Because the truth and MemPrint pass through the same clock mapping,

Table 1: Medians over programs, each the median of 3 runs. Whole-run errors and peak errors are percentages of the truth (pages touched plus the runtime's pages, on the native clock). The 90th percentile of the absolute error and the lowest peak error show the worst cases. Overhead is the run time over a plain RSS run (×). Data $\hat{f}_1$ is the data footprint, $\max(A-F,0)+M+B$, and Charged $\hat{f}_2$ is the charged footprint, $\max(P-F,0)$ (Section 3).

			Whole-run error (%)			Peak error (%)		Overhead
Programs	Estimate	n	signed	absolute	q90 abs.	median	lowest	(×)
Allocator-heavy (7)	RSS	7	+16	17	55	+10	0	1.00
	Data $\hat{f}_1$	7	−4	8	10	−2	−4	0.99
	Charged $\hat{f}_2$	7	0	9	23	−1	−3	0.99
	MemPrint	7	−5	5	17	−5	−16	34.4
Others (16)	RSS	16	+1	2	8	0	−4	1.00
	Data $\hat{f}_1$	16	−1	3	8	−1	−5	0.98
	Charged $\hat{f}_2$	16	−5	7	10	−4	−9	0.98
	MemPrint	15	−1	1	3	−1	−2	8.9
All (23)	RSS	23	+4	7	20	+1	−4	1.00
	Data $\hat{f}_1$	23	−1	5	9	−1	−5	0.99
	Charged $\hat{f}_2$	23	−4	8	12	−3	−9	0.99
	MemPrint	22	−1	2	12	−1	−16	12.0

MemPrint's timing error against the truth is smaller than its timing error against the real run. Its cost is the wall time of the W5 run and of the counter run over the wall time of an earlier plain RSS run. MemPrint has no W5 curve for darknet, so its rows cover 22 programs.

5 Accuracy on the job's own input

Allocator-heavy programs. The data estimate reduces the median whole-run error of RSS from +16% to −4% and the median absolute error from 17% to 8% (Table 1). MemPrint has a median signed error of −5% and a median absolute error of 5%. Per program (Table 2), the data estimate reduces miniVite's error from +102% to −7%, CPython's from +25% to −3%, and Lua's from +20% to −1%, and its absolute error is at most 11% on every allocator-heavy program. Its peak is within 4% of the truth's on all 7 programs, with a median of −2%, while the RSS peak is 10% too high at the median. MemPrint's lowest peak error is −16%, on Lua.

The charged estimate. The charged estimate has a median signed error of 0% on the allocator-heavy programs, and it lies above the truth on miniVite (+37%), `cc1` (+12%) and Quicksilver (+8%), because it keeps the pages of libraries that only this job maps. Unlike the data estimate, the charged estimate counts memory that the truth does not count, so its error against the truth does not measure its accuracy as an estimate of a node's memory use.

Other programs. On the 16 other programs, RSS is already accurate, with a median signed error of +1% and a median absolute error of 2%. The data estimate has −1% and 3%, and MemPrint has −1% and 1%.

The truth's clock. An earlier version of this comparison kept the truth on Pin's count of memory references. Every estimate then had whole-run errors of 10% or more in the same direction on 7 of the 16 other programs, from +10% to +35% on the GAP programs and from −10% to −25% on atax, NPB-IS and NPB-MG, with peak errors within 6%. A phase that executes few memory references per second, such as GAP's graph construction, takes a smaller share of the run

Table 2: Whole-run error per program (% of the truth), signed and absolute, median of 3 runs. Overhead is the run time over a plain RSS run (×). The model's two estimates come from the same run. MemPrint has no W5 run of darknet.

	RSS		Data $\hat{f}_1$		Charged $\hat{f}_2$		MemPrint		Overhead (×)	
Program	sign.	abs.	sign.	abs.	sign.	abs.	sign.	abs.	model	MemPrint
Allocator-heavy										
miniVite	+102	102	−7	9	+37	38	−19	19	0.99	40.2
python	+25	25	−3	8	0	5	−3	3	1.00	27.3
lua	+20	21	−1	5	−4	7	−16	16	0.98	23.6
sqlite	+9	13	+1	11	0	11	+1	4	0.96	50.3
perl	+11	11	−9	9	−8	9	−13	13	0.99	34.4
quicksilver	+16	17	−4	4	+8	9	−5	5	0.99	18.7
cc1	+15	15	−5	5	+12	12	−4	4	1.00	93.0
Others										
2mm	−1	2	0	1	−4	4	0	0	1.01	5.9
atax	−6	7	−6	7	−12	12	−1	1	0.60	23.1
darknet	+1	1	−8	9	−9	9	–	–	1.21	–
gap-bfs	+8	10	+3	7	−1	9	−3	3	0.98	8.9
gap-cc	+5	7	−1	5	−5	8	−3	3	0.96	8.5
gap-pr	+5	7	−1	6	−5	9	−3	3	0.97	7.4
gap-sssp	+4	7	+5	9	+3	10	−3	3	1.04	8.3
gemm	−1	1	−1	1	−5	5	−1	1	1.02	5.6
hpccg	0	1	−1	1	−3	3	−1	1	0.98	12.2
jacobi-2d	−2	2	−1	1	−6	6	−1	1	1.00	4.4
kripke	+1	2	0	1	0	1	0	0	1.01	11.5
lulesh	+6	9	+4	7	+2	8	−1	1	1.00	11.3
minife	+3	4	−1	1	+1	2	−1	1	0.99	13.9
npb-cg	+1	2	−4	4	−9	9	−2	2	0.96	11.9
npb-is	0	1	−2	2	−5	5	−1	1	0.98	5.6
npb-mg	−2	2	−3	3	−6	6	−1	1	0.84	22.4

on Pin's clock than on the wall clock. Placing the truth on the native wall clock removes these errors, and the wall clock is the axis on which a scheduler observes a job.

Overhead. The model's run takes 0.99× the time of a plain RSS run at the median, and 0.96× to 1.04× on 20 of the 23 programs. The plain RSS run includes the 40 ms that the external sampler takes to start, which explains the ratios of 0.60× for atax and 0.84× for NPB-MG, whose runs last 0.10 s and 0.31 s. darknet runs 1.21× slower under the model in all 3 runs. A thread that sleeps for 0.99 s between samples slows darknet by the same 1.22×, so the cost comes from the presence of a second thread in a single-threaded program, and not from the sampling. We did not identify which code path the second thread slows down. The mode that reads `smaps` takes 0.98× the time of a plain RSS run at the median (0.60× to 1.18×). MemPrint's W5 run with its counter run takes 12.0× a native run at the median, 8.9× on the other programs and 34.4× on the allocator-heavy programs, with 93.0× on `cc1`.

5.1 Where the remaining RSS goes

Table 3 shows the size of each part of the model's run against the truth. On the allocator-heavy programs, glibc holds free memory equal to 6% to 39% of the truth on 5 of the 7 programs, and

Table 3: Parts of the model's run, each summed over the run as a percentage of the truth's sum (the truth is 100). Data is estimate 1. Free is glibc's free bytes. File RSS and file PSS are the RSS and PSS of file-backed pages (RSS minus anonymous RSS, and PSS minus anonymous RSS). Median of 3 runs.

Program	Data (%)	Free (%)	File RSS (%)	File PSS (%)
miniVite	93	39	88	57
python	97	15	20	9
lua	99	23	6	1
sqlite	101	6	9	4
perl	91	15	10	4
quicksilver	96	2	23	15
cc1	95	2	23	20
others (median of 16)	99	1	6	0

file-backed pages make up 6% to 88% of the truth. On the other programs, both parts are small, at a median of 1% and 6%.

miniVite has the largest share of both parts. glibc's free bytes amount to 39% of the truth, and file-backed pages amount to 88%. A sample of `/proc/pid/smaps` in the middle of a run shows where the file-backed pages are. The MPI library holds 2348 KiB, PMIx 2096 KiB, the C library 1496 KiB and the C++ runtime 1356 KiB, against a truth of 9.7 MiB on average over the run. The PSS of the C library is 77 KiB and that of the C++ runtime 199 KiB, because other processes map the same pages, but the PSS of the MPI and PMIx libraries equals their RSS, because only miniVite maps them. Subtracting glibc's free bytes alone therefore reduces miniVite's error only from +105% to +66% (Section 11), and leaving out the libraries' pages reduces it to −7%.

6 Transfer to a larger input

A scheduler rarely has a profile of the job's own input, so a footprint model is more useful if a curve measured on small inputs predicts the curve of a larger one. We apply one transfer rule, fixed before looking at the results, to every estimate. The peak at the largest input is a linear regression of the smaller inputs' peaks on the input size [2], never below the largest peak seen, and the shape is the curve of the largest smaller input, scaled to that peak. The largest input is held out, and the model's smaller inputs run once each.

The data estimate transfers as accurately as MemPrint (Table 4). On the allocator-heavy programs, its median whole-run error is −4% (10% absolute), against +20% (22%) for RSS and −6% (10%) for MemPrint. On the other programs, it is −2% (6%), against +2% (10%) for RSS and 0% (5%) for MemPrint.

Three cases show where transfer fails. On `cc1`, every estimate is 226% to 351% too high, because the three inputs are different source files whose size predicts the compiler's memory poorly. On NPB-MG, the data estimate is 67% too low and RSS 96% too low, because the only usable smaller input runs for a few milliseconds and the 5 ms sampling misses its peak, while MemPrint counts every reference and is 15% too high. On 2mm, gemm and jacobi-2d, RSS is 25% to 35% too low for the same reason, because the smallest input's run fits in one sample of the external sampler, which sees only the runtime. The model's thread starts before `main` and samples these runs, and its error on them is −10% to −4%. The transfer of a model trained on short runs therefore depends on how often it samples the start of a run.

Sampling short runs faster. We tested three faster samplers, with a protocol committed

Table 4: Whole-run error (% of the truth), signed and absolute, of each estimate predicted for a program's largest input from its smaller inputs. The peak is a linear regression of the smaller inputs' peaks on the input size, never below the largest of them, and the shape is the largest smaller input's curve. – marks a program whose smaller inputs show nothing above the runtime's pages, for which the scheduler falls back to the user's request.

	RSS		Data $\hat{f}_1$		Charged $\hat{f}_2$		MemPrint	
Program	sign.	abs.	sign.	abs.	sign.	abs.	sign.	abs.
Allocator-heavy								
miniVite	+118	119	+36	45	+102	104	−6	7
python	+43	43	0	10	+6	11	+2	10
lua	+20	22	−4	6	−4	6	−17	17
sqlite	+12	20	−4	20	−2	19	−2	9
perl	+19	20	−10	10	−7	8	−14	14
quicksilver	+16	21	−6	6	−2	2	−7	7
cc1	+226	227	+248	248	+313	314	+351	351
Others								
2mm	−27	27	−4	8	−1	11	+4	5
atax	–	–	–	–	–	–	+21	21
gap-bfs	+6	10	+1	5	+1	6	−4	6
gap-cc	+5	8	−1	5	−1	5	−5	5
gap-pr	+5	16	−3	19	−3	19	−4	14
gap-sssp	+7	10	+2	6	+1	6	−1	4
gemm	−35	35	−10	11	−7	14	+2	2
hpccg	−1	1	0	1	0	1	−1	1
jacobi-2d	−25	25	−5	7	0	12	0	0
kripke	0	2	0	2	0	2	0	1
lulesh	+5	9	+3	8	+3	8	0	6
minife	+4	6	0	2	+2	5	−2	2
npb-cg	+5	8	−2	2	−2	2	+4	5
npb-is	−8	17	−18	19	−18	19	−14	15
npb-mg	−96	97	−67	68	–	–	+15	28
median, allocator-heavy	+20	22	−4	10	−2	11	−6	10
median, others	+2	10	−2	6	0	6	0	5

before the runs (`paper_rss/sampling/PROTOCOL.md`), and rebuilt the transfer from each (Table 5). The first reads `smaps_rollup` every 1 ms instead of every 5 ms. The second reads the process's RSS counters from `/proc/self/status` every 100 μs, and glibc's free bytes every 1 ms. The third samples on memory growth. A software page-fault event on each CPU (`perf_event_open`, inherited by the program's threads) wakes the sampling thread after every 256 page faults, and the thread also samples every 5 ms. Each sampler ran 3 times on every smaller input. Sampling `smaps_rollup` every 1 ms reduces NPB-MG's error from −50% (in these runs, against −67% in Table 4) to +1% and gemm's from −9% to −3%, and it gives the lowest median error over the 23 programs, 2.1% against 2.8%. The two samplers that read `/proc/self/status` also correct NPB-MG, but they are worse than the 5 ms sampler on gemm (−20% and −23%) and 2mm, because the kernel's RSS counters lag the page tables on short runs. On Linux 4.18, each thread's changes to these counters are cached and added to the process's counters in batches, and the peak RSS that the kernel reports (`VmHWM`) is taken from the same counters. In a test that touches 128 KiB without huge pages, `RssAnon` did not change, while `smaps` counted the 128 KiB. On atax at its smallest input,

Table 5: The model under four samplers: `smaps_rollup` every 5 ms (the model's) and every 1 ms, the RSS counters of `/proc/self/status` every 100 μs, and a sample after every 256 page faults. Transfer as in Table 4, rebuilt from each sampler's runs of the smaller inputs; blind configurations as in Table 11 and Table 12, variant (b). Run time at the largest inputs, 3 runs interleaved with runs of the program alone.

	`smaps` 5 ms	`smaps` 1 ms	`status` 100 μs	faults
Transfer to the largest input, signed whole-run error (%), median of 3 runs				
NPB-MG	−50	+1	+8	−6
2mm	−3	+1	−8	−9
gemm	−9	−3	−20	−23
jacobi-2d	−2	−1	−4	−13
atax	–	−33	−25	−41
Median |error|, 23 programs	2.8	2.1	3.1	3.9
Blind configurations, error (%) of the extrapolation to the largest				
NPB MG	–	+26	+13	+27
NPB IS	–	+17	−7	−8
NPB CG	–	+3	−2	+8
Median |error|, all configurations	–	2.0	1.4	6.5
Cost and samples				
Run time over the program alone (×), median	1.04	1.03	1.04	1.04
largest	1.27	1.25	1.26	1.26
Samples per smaller-input run, median	68	288	1,974	76

a run of a few milliseconds, `smaps_rollup` reads 1.98 MB at the peak, and `VmHWM` is 1.04 MB. The page tables, which `smaps_rollup` walks, are therefore the reference for runs this short, and `VmHWM` is not. All four samplers cost 1.03× to 1.04× the run time of the program alone at the median, and the sampling thread's cost on darknet, 1.25× to 1.27×, is the same for every sampler (Section 5).

7 Scheduling with the model's curves

We place the model's curves in DMSched-Sim, the trace-driven scheduling simulator of the corpus. The simulator replays a synthetic job stream with MetaCentrum's shape (3000 jobs, 1 to 16 cores, run times of 5 to 720 minutes) on 32 nodes with 32 cores and 128 GB each and a 1 TB memory pool. Each job takes the footprint curve of a random corpus program with a smaller input, scaled to the job's peak, and the scheduler reserves memory by the oracle's estimate of that curve. The reservation divides the run into 20 windows, holds the estimate's largest value in each window, widens each window to its neighbours, and adds a margin times the peak. A job whose memory exceeds its reservation is killed, loses its work, and is requeued with a reservation 1.5 times larger. Each oracle's margin is calibrated by leaving one program out, so a job of program p gets the 90th percentile of the margins the other programs need to never fall below their truth.

On the pages truth (Table 6), the data estimate gives the lowest mean response time of all estimates that profile the job's own input, 433 minutes at a load of 0.8, against 515 for the RSS curve, 519 for MemPrint on the native clock and 479 for the exact peak. It also has the highest throughput of these estimates (44.9 jobs per hour) and the fewest kills (319). Its calibrated margin is 7%, against 12% for MemPrint, because no program needs a margin above 9% with it. MemPrint on Pin's own clock gives 1607 minutes, because its curve and the job's footprint disagree on when memory is used.

Every transferred estimate except MemPrint on Pin's clock gives a response time at least twice

Table 6: Scheduling simulation under the pages-touched truth on the native clock: mean response time (minutes) at three offered loads, and throughput (jobs per hour), OOM kills and core-hours lost to them at a load of 0.8. Means over 3 seeds of 3000 jobs on 32 nodes. Every estimate's margin is the 90th percentile of the margins the other programs need (leave one program out), and the user's request takes 10%. The second block profiles the job's own input, and the third transfers from the program's smaller inputs.

	Mean response (min)			At load 0.8		
Oracle	0.6	0.8	1.0	jobs/h	kills	lost core-h
Exact curve (truth)	55	94	261	62.3	0	0
Exact peak	172	479	698	47.4	0	0
User request	1867	2244	2452	23.6	0	0
KS+	491	976	1233	33.7	9236	5012
RSS peak	532	875	1126	35.4	412	886
RSS curve	185	515	776	41.3	407	1732
MemPrint W5	1247	1607	1797	27.2	386	1363
MemPrint W5, native clock	197	519	707	42.6	384	593
Data $\hat{f}_1$	159	433	644	44.9	319	1888
Charged $\hat{f}_2$	192	508	691	41.7	332	1916
RSS peak	1913	2269	2476	22.3	418	7
RSS curve	2369	2632	2869	20.3	698	553
MemPrint W5	1890	2273	2475	22.3	387	1363
MemPrint W5, native clock	911	1283	1477	30.5	275	699
Data $\hat{f}_1$	2523	2806	3142	20.4	926	570
Charged $\hat{f}_2$	2017	2479	2665	21.0	939	1554

that of the same estimate on the job's own input. MemPrint on the native clock transfers best (1283 minutes), and the data estimate transfers worst (2806 minutes), although its median transfer error equals MemPrint's (Section 6). The calibrated margin follows the worst programs, and the data estimate needs a margin of 1.00 on gemm and 2.67 on NPB-MG, the programs whose smaller inputs are too short for 5 ms sampling, so its 90th percentile is 0.53, against 0.23 for MemPrint.

Under the RSS truth, which the kernel charges and OOM-kills on, the RSS curve of an earlier run is the best estimate (393 minutes at a load of 0.8), because it measures the truth itself. The data estimate gives 748 minutes, close to MemPrint on the native clock (699 minutes), because it leaves out the retained memory and libraries that the RSS truth counts, and its calibrated margin must cover them. The estimate a scheduler should use therefore depends on what the node enforces. A limit on RSS calls for the RSS curve, and a limit on the memory the job uses, as in a cluster whose allocators return freed memory, calls for the data estimate.

8 Other allocators and transparent huge pages

We ran the 7 allocator-heavy programs under jemalloc 5 and TCMalloc (gperftools 2.18, `tcmalloc_minimal`), loaded with `LD_PRELOAD`, and measured the truth with Pin under the same allocator (Table 7).

TCMalloc. RSS overestimates the footprint by a median of 46%, more than under glibc, because TCMalloc keeps freed spans in its page heap and its caches. The data estimate has a median error of −4%, and it is low on SQLite (−22%) and miniVite (−16%). Without the allocator's free bytes, the estimate is +21%, so the statistics remove most of the gap.

Table 7: Signed whole-run error (% of the truth under the same allocator) under jemalloc and TCMalloc, with transparent huge pages enabled (the machine's setting) and disabled for the process. *Anon* is the data estimate without the allocator's free bytes. Median of 3 runs. Overhead is the run time over a plain RSS run under the same allocator.

	THP enabled				THP disabled		
Program	RSS	Anon	Data $\hat{f}_1$	Charged $\hat{f}_2$	RSS	Data $\hat{f}_1$	Charged $\hat{f}_2$
jemalloc							
miniVite	+187	+189	+149	+202	+83	−10	+27
python	+59	+61	+35	+43	+49	+5	+14
lua	+112	+125	+60	+61	+96	+10	+12
sqlite	+29	+49	+21	+23	+26	−4	−1
perl	+26	+39	+22	+26	+25	−1	+4
quicksilver	+56	+59	+58	+69	+23	+4	+16
cc1	+28	+19	+15	+35	+25	−2	+19
median	+56	+59	+35	+43	+26	−1	+14
TCMalloc							
miniVite	+76	+21	−16	+21	+68	−27	+10
python	+48	+32	−3	+6	+47	−5	+4
lua	+62	+55	0	+3	+63	0	+3
sqlite	+46	+39	−22	−18	+46	−24	−20
perl	+17	+10	−7	−3	+19	−8	−2
quicksilver	+23	+2	−2	+10	+21	−3	+10
cc1	+25	+2	−4	+16	+22	−6	+14
median	+46	+21	−4	+6	+46	−6	+4
overhead (×)	1.00	0.98	0.98	0.98	1.00	0.99	0.99

jemalloc. With transparent huge pages enabled, every estimate overestimates jemalloc's footprint, RSS by a median of 56% and the data estimate by 35%, with +149% on miniVite. jemalloc places its extents at 2 MiB aligned addresses, so the kernel backs them with huge pages, and a 2 MiB page counts in full when the program touches 4 KiB of it. In a run of Lua under jemalloc, 45 MiB of the 65 MiB of anonymous memory is in huge pages. We disabled transparent huge pages for the process with `prctl(PR_SET_THP_DISABLE)`, which the library calls in its constructor on request. The data estimate's median error under jemalloc then drops to −1%, and RSS's to +26%. Under TCMalloc and under glibc, disabling transparent huge pages changes the median errors by at most 4 points (Table 8).

The memory that huge pages add is real. A node with transparent huge pages enabled holds the whole 2 MiB page in memory, and the pages-touched truth undercounts it, as MemPrint would. A scheduler on such a node needs the estimate with huge pages, which the model measures and Pin does not, and a scheduler that wants the 4 KiB footprint can disable huge pages for the profiling run. We also checked two other kernel counters that we expected to matter. `LazyFree`, the pages released with `MADV_FREE` that stay resident until the kernel reclaims them, was 0 in the jemalloc runs, and the `Referenced` bytes of llama2.c's model file equal its RSS.

How much huge pages add. From the runs of Table 7, Table 8 and Section 9, we divided each program's peak anonymous RSS with transparent huge pages enabled by its peak with them disabled. Under glibc, the ratio is within 1.02 on 21 of the 23 programs, and huge pages add at most 2.6 MiB, with 1.16× on miniVite and 1.04× on Quicksilver. Under TCMalloc, the median

Table 8: The corpus under glibc with transparent huge pages enabled and disabled for the process: median signed and absolute whole-run error (% of the truth). The data estimate here comes from the mode that reads `smaps`.

	RSS				Data $\hat{f}_1$			
	THP on		THP off		THP on		THP off	
Programs	sign.	abs.	sign.	abs.	sign.	abs.	sign.	abs.
Allocator-heavy (7)	+16	17	+15	15	−3	9	−7	9
Others (16)	+1	2	0	3	−1	3	−1	4

Table 9: Signed whole-run error (% of the pages-touched truth) on the language model engines, median of 3 runs. Data $\hat{f}_1$ without files is $\max(A - F, 0) + B$. The model's overhead is the run time of the mode that reads `smaps` over a plain RSS run, and MemPrint's is its W5 and counter runs over a native run (×).

		Data $\hat{f}_1$				Data $\hat{f}_1$	Overhead (×)	
Engine and input	RSS	no files	with files	Charged $\hat{f}_2$	MemPrint	THP off	model	MemPrint
llama2c 110M	+12	−86	+12	+11	0	0	1.00	3.9
llama2c 15M	0	−92	0	−3	−2	0	0.98	6.0
llama2c 42M	+15	−81	+15	+14	0	−1	1.00	4.4
llamacpp 135m-n128	+1	−39	−1	0	−3	−7	0.99	38.9
llamacpp 135m-n32	−26	−56	−28	−27	−13	−30	0.94	67.4

ratio is 1.02. Under jemalloc, it is 1.17 (14.8 MiB), with 2.09× on miniVite (29.5 MiB). llama2.c's 42M and 110M models have ratios of 4.9× and 5.2×, 24 and 47 MiB of anonymous memory, but their RSS grows only 1.15× and 1.14×, because the mapped model file dominates it.

Which 4 KiB pages were touched. A scheduler that wants the 4 KiB footprint on a node with huge pages needs to know which 4 KiB pages of each huge page the program touched. We mapped 64 MiB, touched one 4 KiB page in each 2 MiB, and read the interfaces that need no privileges (`paper_rss/sampling/thp_test.c`). With huge pages enabled, `smaps` reports 64 MiB of `Rss` and of `AnonHugePages`, and `mincore` reports all 16,384 pages of the range as resident. With `MADV_NOHUGEPAGE` on the range, or with huge pages disabled for the process, all three report 128 KiB. `/proc/pid/pagemap` gives page frame numbers only to a process with `CAP_SYS_ADMIN`, and the idle page tracking of `/sys/kernel/mm/page_idle` needs root. Without privileges, the 4 KiB footprint under huge pages is therefore measurable only by preventing huge pages in the profiling run, with `MADV_NOHUGEPAGE` or `PR_SET_THP_DISABLE`.

9 Language model engines

llama2.c maps its model file and reads the weights from the mapping, and llama.cpp maps its GGUF file and copies part of it into buffers it allocates. Without the data-file term, the data estimate is 81% to 92% too low on llama2.c and 39% to 56% too low on llama.cpp (Table 9), because the weights are file-backed pages. With the term, the data estimate is within −1% to +15% of the truth on llama2.c and on llama.cpp generating 128 tokens. On llama2.c, the remaining overestimate comes from transparent huge pages. The engine allocates its key and value caches for the whole context of 1024 positions with `calloc` and fills only the positions it generates, and 45 MiB of its 47.5 MiB of anonymous memory is in huge pages. With huge pages disabled, the data estimate is within 1% of the truth on all three model sizes.

llama.cpp generating 32 tokens is the exception. Every estimate of the model is 26% to 28% too low, against 13% for MemPrint. The run lasts less than a second, and most of it loads the model, so the estimates and the truth disagree on when the model's pages become resident. The model's run costs 0.94× to 1.00× a plain RSS run, and MemPrint's W5 costs 3.9× to 6.0× on llama2.c and 38.9× to 67.4× on llama.cpp.

10 Programs and inputs not seen before

A companion study [16] asks how MemPrint can estimate the footprint of a program that has never been traced, because MemPrint's sampled estimate needs a correction factor that is trained on full Pin traces of the same program. It answers from source code, with an abstract interpreter that predicts the access counts of every byte. The footprint model needs no such training, because it measures the job's own run, and its only per-program quantity is the runtime's pages B, which Section 3 takes from an earlier plain run. This section asks what the model needs for a program that has not run before, in three settings: a job that is measured in its own run, the blind programs of the companion study, and a job whose larger input is predicted with the growth of other programs.

10.1 The runtime's pages from the job's own run

Table 10: Estimate 1 with B from an earlier plain run (the smallest RSS) and from the model's own run (the file-backed RSS at the first sample, after the shared libraries are loaded). Median over programs of the median of 3 runs; whole-run errors and the peak error in % of the truth, whose B stays the earlier run's.

	B from an earlier run			B from the same run		
Programs	signed	absolute	peak	signed	absolute	peak
Allocator-heavy (7)	−3.7	7.9	−2.5	+1.5	6.8	−0.3
Others (16)	−1.0	3.4	−1.3	+0.2	1.7	−0.0
All (23)	−1.3	5.1	−1.3	+0.6	4.4	−0.1
miniVite	−6.6	9.0	+3.9	+30.2	30.4	+25.9

A job can supply B itself. The library samples the process from its constructor, which runs after the dynamic loader has mapped the shared libraries and before `main`, so its first sample holds the runtime's pages and almost none of the program's. We take B as the file-backed RSS of that sample, RSS minus the anonymous RSS, because the anonymous pages of that moment are already in $A(t)$. This B is 1.5 to 5.7 MB, against 1.2 to 2.6 MB for the smallest RSS of the plain sampler, which reads the process right after `exec`, before the libraries are loaded. The median whole-run error moves from −1.3% to +0.6% and the absolute error from 5.1% to 4.4% (Table 10). The exception is miniVite, at +30%, because its MPI and OpenMP libraries hold about 4 MB of file-backed pages against a 14 MB footprint. Whether the pages of a job's own libraries belong in its footprint is a choice of definition, and with B from the same run the model needs no run of the program before the job.

10.2 The companion study's blind programs

The companion study tested its interpreter on HPCCG and LULESH, and then on NPB IS, CG and MG, with predictions committed before the programs were traced. We ran the footprint model on the same configurations, three times each, sampled every 1 ms because many configurations last

Table 11: The model on the companion study's blind configurations [16], against its Pin truth (the distinct bytes touched over the whole run, MB): the measured footprint (the largest $A - F$ of the run, median of 3 runs, MB), its error alone (a) and with a runtime constant fitted on the other programs (b), and the companion study's prediction from source, in % of the truth.

		Truth	Measured	Model (%)		From source
Program	Config	(MB)	(MB)	(a)	(b)	(%)
HPCCG	10	0.81	0.37	−54.5	−0.6	+2.1
	12	1.08	0.67	−37.5	+3.3	+1.6
	14	1.45	0.98	−32.4	−2.2	+1.2
	16	1.95	1.55	−20.4	+2.0	+0.9
	20	3.42	2.95	−13.9	−1.1	+0.5
	24	5.64	5.27	−6.5	+1.3	+0.3
	28	8.77	8.29	−5.5	−0.5	+0.1
LULESH	5	0.58	0.13	−77.8	−8.6	−2.7
	8	0.87	0.43	−50.5	−4.0	−2.3
	10	1.25	0.82	−34.4	−2.0	−4.1
	12	1.76	1.37	−22.0	+0.9	−2.7
	15	4.22	2.43	−42.4	−32.8	−1.1
	18	6.87	4.25	−38.1	−32.2	−0.4
	20	9.23	5.82	−37.0	−32.6	−0.5
NPB IS	S	1.00	0.64	−36.0	+8.6	+0.3
	W	9.12	8.76	−3.9	+1.0	+0.0
NPB CG	S	1.74	1.30	−25.4	−0.5	+0.1
	W	8.38	7.94	−5.3	−0.2	+0.0
NPB MG	S	1.52	1.14	−24.8	+4.5	−7.3
	W	58.53	58.17	−0.6	+0.1	−9.1

a few milliseconds, with one more sample when the process exits. The protocol was committed before the runs (`paper_rss/blind/PROTOCOL.md`). The truths were published before this test, but the model has no parameter that is fitted to these programs, except a runtime constant that is fitted on the other programs.

Two definitions of the footprint. The companion study's truth counts the distinct bytes that the run touches from start to end, and it includes every byte that the runtime touches, read-only library data among them. The model measures the anonymous memory in use, so it leaves out the runtime's read-only data. We therefore score two variants. Variant (a) is the largest $A - F$ of the run alone. Variant (b) adds a constant R, the median of the truth minus the measured footprint over the configurations of the other programs, which is 0.40 to 0.44 MB. With R, the model is within a median of 2.0% of the truth over the 20 configurations, against 1.0% for the prediction from source (Table 11). On NPB MG, where the prediction from source fails (-7.3% and -9.1%), the model is within $+4.5\%$ and $+0.1\%$.

LULESH at $s = 15$, 18 and 20 is the exception, at -32% to -33%. Its temporary arrays of 8 values per element (216 KB at $s = 15$) cross glibc's 128 KiB threshold for `mmap` between $s = 12$ and $s = 15$, so that each cycle can receive fresh pages, and a count of every byte that the run touches grows faster than the memory in use (the truth grows 2.4× from $s = 12$ to 15, while the input grows 1.95×). We did not trace this behaviour, so it remains the likely cause. The model measures the memory in use, which is what a scheduler reserves, and on our pages-in-use truth LULESH at $s = 60$ is within $+4\%$ (Table 2).

Table 12: Transfer on the blind programs: the largest configuration with a truth (EXTRA) or the middle one (INTER) predicted from the others' measured footprints by the rule of Section 6, plus R, in % of the truth; with the floor taken from the smaller configurations only (post hoc); the model measured on that configuration; and the prediction from source.

Program	Split	Config	Transfer	Post hoc	Measured	From source
HPCCG	EXTRA	28	+0.0	+0.0	−0.5	+0.1
HPCCG	INTER	16	+346.9	+0.8	+2.0	+0.9
LULESH	EXTRA	20	−32.9	−32.9	−32.6	−0.5
LULESH	INTER	12	+253.7	−3.3	+0.9	−2.7
NPB IS	EXTRA	W	+16.7	+16.7	+1.0	+0.0
NPB CG	EXTRA	W	+3.3	+3.3	−0.2	+0.0
NPB MG	EXTRA	W	+25.6	+25.6	+0.1	−9.1

Transfer. The companion study also predicts a held-out configuration from a program's other configurations, the largest (EXTRA) and the middle one (INTER). The rule of Section 6, a linear fit of the peak on the input size that is never below the largest peak seen, predicts HPCCG's largest grid within 0.02% (Table 12). For the middle configuration, the rule as committed keeps the floor at the larger configurations' peaks, which gives $+347\%$ and $+254\%$. The rule was written for extrapolation, and with the floor taken from the smaller configurations only, which we decided after seeing these results, the middle configurations are within $+0.8\%$ and -3.3%. NPB has only classes S and W with a truth, so W is predicted from S alone in proportion to the size, which gives $+3\%$ on CG, $+17\%$ on IS and $+26\%$ on MG. A measured footprint is therefore more accurate than a transferred one whenever the job can be run under the model, and the transfer needs more than one smaller input. Under the samplers that read `/proc/self/status`, the extrapolations of NPB IS and MG are -7% to $+27\%$ (Table 5), but the configurations below 1 MB have larger errors, up to $+36\%$ on LULESH at $s = 5$ and -22% on HPCCG at 10^3, against -9% and -1% for `smaps_rollup` every 1 ms.

10.3 Borrowing the growth of other programs

Table 13: A program's largest input predicted from one run at a smaller input, scaled by the input sizes to the power β, with β chosen by each method: the median and 90th percentile of the absolute whole-run error (% of the truth) and the number of the 22 programs within 10%. The method marked as an oracle chooses after the fact, and the last row uses all of the program's own inputs. Left: the run at the smallest input, as committed; right: at the largest smaller input (post hoc).

	Smallest input			Largest smaller input		
Method	median	q90	$\leq 10\%$	median	q90	$\leq 10\%$
Own transfer (two smaller inputs)	3.9	36	16	3.9	36	16
Proportional ($\beta = 1$)	12.4	96	11	4.1	65	12
Uniform (mean β)	94.5	153	0	7.6	70	13
Nearest program's β	34.0	96	7	9.5	61	12
RBF mixture of β	26.9	96	8	11.1	62	11
Best known β (oracle)	10.6	96	10	2.2	56	16
The program's own β (all its inputs)	8.5	95	15	3.8	34	14
Spearman ρ of distance and error, median	0.36			-0.11		

The companion study's second family of methods borrows the model of a known program that is similar to the new one, with similarity measured by a descriptor of memory behaviour [16]. The analogue for the footprint model is a program that has run once, at a small input, and whose larger input is to be predicted without its growth being known. We predict the largest input's curve as the small run's curve scaled to the peak $p_S(x_L/x_S)^\beta$, where x is the input size, and we borrow the exponent β from the programs that are known. Each known program's β is the slope of its log peak against its log input size over all its inputs. The nearest-program method takes the β of the program closest in a descriptor of the small run, made of six standardised features: the allocator's share of RSS, the mean over the peak, the time of the peak, the share of the run near the peak, the value at a tenth of the run, and the final value. The RBF method weights every known β by its distance, as the companion study does. We evaluate with each of the 22 programs that have a smaller input held out in turn (`paper_rss/borrow.py`, committed before the results).

Borrowing does not beat the exponent 1 (Table 13). From the smallest input, the proportional prediction has a median absolute error of 12.4%, against 34.0% for the nearest program's β and 26.9% for the RBF mixture. The descriptor ranks the known programs by how well their β transfers with a median Spearman ρ of 0.36, against 0.89 for the companion study's descriptor on its example. The smallest inputs of the four PolyBench programs hold less data than the runtime's pages vary, so every method misses them by 94% to 96%, and their own β comes out near 3. With the run at the largest smaller input instead, which we chose after seeing these results, the proportional prediction has a median of 4.1%, against 9.5% and 11.1% for borrowing, and ρ is -0.11. Most programs grow with an exponent near 1 in their input size, because the input sizes we use count the data, and the programs that do not, `cc1` (0.42), Quicksilver (0.65) and miniVite (0.78), are not closer to each other in the descriptor than to the others. The companion study's descriptor is a spectrum of access counts, which determines its correction factor directly, whereas the shape of a footprint curve does not determine how the footprint grows with the input. The transfer from two smaller inputs of the same program (Section 6) has a median of 3.9% and is the better use of two cheap runs.

10.4 Similarity of the code instead of the run

Table 14: Code similarity and the model's terms, over the 22 programs with code of their own (all but `cc1`). ρ: Spearman correlation over program pairs of the code distance and the difference in the term (Mantel test, 10,000 permutations; in parentheses when $p \geq 0.05$). R': median absolute error (MB) of the term taken from the nearest program, each program held out in turn. Borrowed β: the absolute whole-run error (% of the truth) of the prediction of Table 13 from the largest smaller input, with β from the nearest program, for the 21 programs with a smaller input: median, 90th percentile and the number within 10%. Memory code: the lines that allocate, free or grow memory, with 10 lines on each side.

	ρ				R'	Borrowed β (%)		
Space	B	r	R'	β	(MB)	median	q90	$\leq 10\%$
Median of the others; $\beta = 1$	–	–	–	–	1.32	3.8	45	12
Static features, all	(0.13)	0.67	0.54	0.56	0.44	5.7	24	13
source only	(0.05)	0.56	0.32	0.42	0.52	4.3	20	14
executable only	0.24	0.54	0.59	0.52	0.58	5.6	20	13
Run descriptor (Table 13)				(−0.15)		9.7	44	11
gte-modernbert (149M)	(0.10)	0.55	0.36	(0.05)	0.70	4.1	47	14
bge-code-v1 (1.5B)	(0.03)	0.39	0.28	(−0.09)	1.11	4.4	25	13
Qodo-Embed-1 (1.5B)	(0.10)	0.47	0.37	(−0.02)	0.75	5.7	47	13
Qwen3-Embedding (8B)	(0.11)	0.44	0.29	(0.03)	0.75	5.7	45	13
gte-modernbert (149M), memory code	(0.07)	0.49	0.40	(−0.03)	3.87	5.9	47	13
bge-code-v1 (1.5B), memory code	(0.01)	0.37	0.27	(−0.12)	0.76	5.9	25	12
Qodo-Embed-1 (1.5B), memory code	(0.07)	0.44	0.39	(−0.06)	2.31	8.3	45	12
Qwen3-Embedding (8B), memory code	(0.12)	0.44	0.42	(0.02)	0.76	7.9	45	12
The program's own β						3.8	26	14

A program that has not run can still be compared with known programs by its code, without the companion study's interpreter. We test whether programs with similar code have similar terms: the runtime's pages B, the allocator's free share r, the pages of libraries R' (Section 10) and the growth exponent β. The code is the program's sources, or the script for the four programs run by an interpreter (`paper_rss/similar_src.py`). We compare two kinds of code space. The first is 21 static features, counted on the code and the executable without running either: the lines of code, the rates of allocations, frees, reallocations, growing containers, node-based containers, vectors, smart pointers, OpenMP pragmas and MPI calls, the share of allocations inside a loop, and the executable's text, data and bss sizes, its shared libraries and their total size. The second is the code embeddings of four open models for code retrieval, run locally: gte-modernbert-base, bge-code-v1, Qodo-Embed-1-1.5B and Qwen3-Embedding-8B. Each model embeds the code in windows of 512 tokens, and a program's embedding is the mean of its windows. Because most of a program's code computes rather than allocates, we also embed only its memory code, the lines that match the allocation patterns of the static features with 10 lines on each side, a median of 16% of the compiled programs' code (2% for NPB, which allocates its arrays statically, to 48% for miniVite) (`paper_rss/similar.py`, `similar_embed.py`). We chose the features, the models and the memory code after the borrowing results, so this comparison is exploratory.

The static features predict the terms better than every embedding (Table 14). Their distance correlates with the difference in r ($\rho = 0.67$), R' (0.54) and β (0.56), and taking R' from the nearest program in them has a median error of 0.44 MB, against 1.32 MB for the median of the other programs. The executable's features alone carry most of this, because r and R' separate the

programs that a language interpreter runs, and the programs that link large libraries, from the rest. The embeddings correlate with r and R' more weakly, and not with β (-0.12 to 0.05, none significant). Neither the memory code nor the larger model changes this: Qwen3-Embedding-8B has $\rho = 0.03$ on β on the whole code and 0.02 on the memory code. An embedding's nearest program is almost always of the same suite (GAP, NPB or PolyBench), so an embedding finds the program's family, but not how its memory grows with the input.

Borrowing β from the nearest program in the static features reduces the 90th percentile of the whole-run error from 45% to 20% to 24%, but not the median, 4.3% to 5.7% against 3.8% for $\beta = 1$. The change in the 90th percentile comes from one program: miniVite, which grows more slowly than its input ($\beta = 0.78$), borrows Quicksilver's β of 0.65, and its error falls from +45% to +17%. With all static features, MiniFE borrows the same β and its error grows from +3% to −24%. The two largest errors, atax (−94%) and NPB MG (−67%), are the same for every β. The program's own β has a 90th percentile of 26%, above the static features' 20% to 24%, which also shows that the difference rests on a few programs. Code similarity thus has a measurable relation to the terms, but over 22 programs it does not predict β more accurately than $\beta = 1$, and the model's own run at the job's input remains more accurate.

10.5 A prediction with no run of the program

Table 15: Predictions with no run of the program: the companion study's interpreter on the source at the largest input, plus the runtime's pages as the other programs' runs show them. Interpreter: its time over the plain native run's (×). Peak error and signed whole-run error in % of the truth, for the prediction with no run, the model's own run, the model's transfer from the smaller inputs and MemPrint; and the peak error of the prediction against the plain run's RSS. The C++ interpreter has no timeline mode, and PolyBench's timeline mode needs more than 16 GB except on atax, so these predictions hold the footprint over the whole run.

	Interpreter	Peak error (%)				Whole-run (%)		RSS peak
Program	(×)	no run	model	transfer	MemPrint	no run	model	no run (%)
2mm	1	+0.1	−0.9	+4.2	−0.4	+4.0	+0.0	+5.7
atax	112	+0.2	−1.0	–	−0.5	+18.1	−4.7	+6.7
gemm	2	+1.1	−1.2	+0.2	−0.6	+1.1	−1.3	+7.3
jacobi-2d	1	+0.6	−1.3	+1.2	−0.6	+0.6	−1.3	+7.5
GAP bfs	87	+45.3	−1.5	−1.5	−0.9	+197.9	+1.3	+45.7
GAP cc	194	+45.5	−1.5	−1.5	−0.8	+197.6	+0.3	+45.9
GAP pr	944	+45.2	−1.5	−1.5	−0.9	+194.7	−0.6	+45.9
GAP sssp	143	+40.2	−6.8	−9.9	−0.5	+205.7	+5.4	+47.6
LULESH	1,340	+57.3	−0.7	−0.7	−0.4	+147.3	+4.0	+58.0
NPB MG	211	−10.3	−1.6	−70.6	−0.9	−0.0	−2.4	−7.9

A program that has never run can be predicted only from its source. The companion study's interpreter runs C and C++ programs from `main` and reports the bytes that they touch [16], and the footprint model adds the terms that the interpreter does not see from other programs' runs. We predict the footprint as the interpreter's footprint P plus R, the median of the other programs' runtime pages B, and the RSS as $P/(1 - r) + R'$, where r is the median share of the anonymous memory that the other programs' allocators hold free at their peak, and R' is the median of their peak RSS minus their largest data estimate, the pages of their libraries (`paper_rss/zero.py`). Every term comes from other programs, and the program itself never runs. We first learned R as the median of the truth minus P over the other programs, which absorbed the interpreter's own

errors on those programs and gave $R = -15$ MB, and we replaced it with the definition above after seeing that result.

The interpreter runs 10 of the 23 corpus programs at the largest input: the four PolyBench kernels, GAP's bfs, cc, pr and sssp at scale 18, LULESH at $s = 60$ and NPB MG. HPCCG at 64^3 exceeds the interpreter's recursion depth, with a limit of 200,000 frames and a 512 MB stack, although it ran at the companion study's 28^3. NPB IS and CG at class A are out of reach, because their inputs come from a chain of random-number calls that the interpreter runs one at a time, 4 hours already at class W. The other programs (the interpreters, `cc1`, darknet, miniVite, Kripke, MiniFE and Quicksilver) use libraries, MPI or input files that it does not run. GAP's spectra are the companion study's, for one trial against the corpus's four or eight, and MG's is at class W, the corpus's input.

On the four PolyBench kernels, the prediction with no run is within 0.1% to 1.1% of the peak (Table 15), as close as the model's own run, because these programs allocate their arrays once and touch them, so the bytes that the interpreter counts are the pages in use. R is 1.7 to 1.9 MB. On GAP, it is 40% to 45% too high: the interpreter counts every byte that the run touches, and GAP's graph builder fills temporary edge lists that it frees before the kernel runs. LULESH is 57% too high for the same reason, because it allocates and frees its temporary arrays of 8 values per element in every step (Section 10). On NPB MG, it is 10.5% low, the interpreter's own error on MG. Because a C++ prediction has no timeline, it holds the peak over the whole run, which reserves 194% to 206% too much on GAP and 147% on LULESH over the run, whose footprint is low for most of it; atax's timeline gives +18% against +35% for its own peak held flat. The allocator term r is 0.4% to 0.5%, because the other programs of this set allocate their arrays once, and the RSS prediction is 5% to 7% above the RSS on PolyBench, the pages of libraries that these kernels do not load.

The interpreter costs 1 to 1,340 times the native run, 1.9 hours for LULESH, against 0.99× for the model's own run. A prediction with no run is therefore useful where the scheduler must place a job before it can run it at all, and only for array programs whose source the interpreter can execute. For every other program, the model's own run, or its transfer from two smaller runs, is cheaper and more accurate.

11 Other ways to correct RSS

Before we settled on the model, we tested four alternatives on the allocator-heavy programs under glibc, each on its own (Table 16). The first, RSS minus glibc's free bytes, is the first half of the model.

Releasing freed memory during the run. Instead of subtracting free bytes, the program can return them to the kernel, so that RSS drops when the program frees memory. We tested two ways. The first calls `malloc_trim(0)` every 5 ms from a thread in the process, which releases the free pages inside every arena with `madvise`. The second sets glibc's trim threshold to 0, which releases free memory at the top of the heap after every `free`. Both reduce the median error less than subtracting free bytes, +11% and +14% against +9%. A trim threshold of 0 releases only the top of the heap, which explains why it changes Lua's error by only 3 points. Both alternatives also change the program's allocation behaviour, and the trim thread costs up to 1.10× on Perl.

Language runtime statistics. For CPython, we sampled the free bytes of its object allocator (the arena bytes minus the bytes in allocated blocks, from `sys._debugmallocstats`) from a thread started in `sitecustomize`. For Lua, we sampled the heap size the garbage collector reports (`collectgarbage("count")`) from a debug hook every 10^6 virtual machine instructions.

Table 16: Signed whole-run error (% of the truth) of each alternative used alone on the allocator-heavy programs, median of 3 runs, and its overhead over a plain RSS run (×). Runtime statistics exist only for CPython and Lua. These runs keep the file-backed pages in every estimate.

Program	RSS	RSS−free	trim 5 ms	trim thr. 0	runtime stats	referenced
miniVite	+105	+66	+58	+55	–	−70
python	+25	+9	+11	+12	+5	+11
lua	+20	0	+2	+17	−19	+2
sqlite	+9	+4	+7	+9	–	−24
perl	+11	−3	+6	+7	–	−16
quicksilver	+17	+16	+14	+14	–	−55
cc1	+16	+15	+15	+15	–	−68
median signed	+17	+9	+11	+14	−7	−24
median absolute	18	10	11	15	35	36
overhead (×)	1.00	0.99	1.03	1.02	0.96	1.02

CPython's whole-run signed error drops to +5%, but its absolute error rises, which means that the estimate is too low in some parts of the run and too high in others. We did not determine why. Lua's heap count leaves out the interpreter's own memory and gives −19%. The statistics exist only for these two programs, and GCC reports its heap only at the end of a compilation.

Referenced pages per window. The kernel can clear the referenced bits of a process's pages (`/proc/pid/clear_refs`), after which `smaps_rollup` counts the pages the process has touched since. We cleared the bits 20 times per run and took the pages referenced in each window as the footprint of that window, in the manner of a working set [4]. The estimate falls below the truth by up to 70% (miniVite), 68% (`cc1`) and 55% (Quicksilver), because a page that the program holds but does not touch within a window is not counted. A scheduler that reserved this estimate would run these programs out of memory.

12 Returning free memory at run time

The model's free bytes are memory that the program has freed and that the allocator keeps resident. A runtime that reads the same statistics can return this memory to the kernel while the program runs, which lowers the RSS and the memory that a system must provision for the job. We added the model to libmpr, the runtime of our allocation-resizing study, which serves each allocation of at least 64 KiB from a per-site reservation that MemPrint profiles and commits only the predicted part of the block. libmpr leaves smaller blocks to glibc, so these blocks hold the free bytes that the model subtracts.

Mechanism. With a share s set (`MPR_TRIM`), libmpr compares glibc's resident free bytes with $\max(s \cdot A, 1\,\text{MB})$, where A is the anonymous RSS that the kernel keeps as a counter in `/proc/self/status`, and it calls `malloc_trim(0)` when the free bytes exceed this threshold. `malloc_trim` releases the whole pages inside every free chunk of every arena with `madvise`. glibc counts the released bytes as free after the trim, because they are free but no longer resident, so libmpr keeps the free bytes at the last trim apart as released, and it lowers this count when the free bytes fall below it, which means that the program has reused released memory. The resident free bytes are then the free bytes minus the released bytes.

libmpr makes this check from `free`, which it already intercepts, and not from a thread. A second thread turns off the paths that glibc takes in a single-threaded process, in which `stdio`

Table 17: Memory summed over the workloads under each build, mean of 3 runs: the peak RSS and the time-averaged RSS (MB), RSS integrated over the run (GB s), and the time-averaged reservation of dynamic provisioning, which holds the request for the first sixth of the run and then each sixth's largest RSS (MB). Trimming at $s = 0.1$, from `free`.

	Allocator-heavy (7)				All (29)			
Build	Peak (MB)	Mean (MB)	RSS×time (GB s)	Dynamic (MB)	Peak (MB)	Mean (MB)	RSS×time (GB s)	Dynamic (MB)
glibc	544	360	0.9	478	4,590	3,600	17.1	4,390
glibc + trim	534	344	0.9	466	4,597	3,524	17.3	4,354
libmpr	539	361	1.0	474	3,764	2,979	15.4	3,549
libmpr + trim	530	345	0.9	466	3,749	2,959	15.6	3,532
libmpr, held-out profile	543	363	1.0	478	3,776	2,988	15.9	3,601
libmpr, held-out + trim	533	344	0.9	465	3,766	2,959	15.8	3,588

and `malloc` skip their locks. With a monitoring thread that never trims, darknet, which parses its files one character at a time, runs 1.23× slower, and with the check in `free` it runs at 0.96×. The resident free bytes grow only by the bytes that the program frees, so `free` counts these bytes and makes the next check only after the program has freed the distance between the resident free bytes and the threshold, and at least 1 MB. A program that frees memory in a short burst is therefore trimmed at the end of the burst and not at the next tick of a timer. A thread is still available for the monitor that writes the model's estimate every 5 ms, and it trims as well.

Methodology. We ran the 29 workloads of the resizing study: the 23 programs at their largest inputs, with miniVite built with the resizing compiler pass, and six configurations of llama2.c and llama.cpp. Each workload ran under six builds, glibc, libmpr with the reservations of its own input and libmpr with reservations profiled on other inputs, each without and with trimming at $s = 0.1$, three times each, with RSS sampled every 20 ms. Four workloads ran at a time on the 32-core machine, which adds about 5% of noise to each run time. Every run produced the output of the glibc run. The libmpr of these runs read only part of llama.cpp's longest reservation lines, which lowered its commit by 0.8 to 4.2 MB and left its RSS unchanged (Section 13). Trimming acts on RSS, so we keep these runs.

Memory. Trimming lowers the time-averaged RSS of the allocator-heavy programs from 360 to 344 MB in total (4.4%), and that of all 29 workloads from 3,600 to 3,524 MB under glibc (2.1%) and from 2,979 to 2,959 MB under libmpr (0.7%) (Table 17). The largest reductions are on miniVite (23 to 18 MB), Lua (45 to 39 MB), CPython (52 to 48 MB) and llama.cpp under glibc (203 to 178 MB at 32 tokens) (Table 18). Under libmpr, llama.cpp gains less from trimming, because libmpr already serves its large buffers and glibc holds less free memory. Trimming does not lower the summed peak RSS, 4,590 against 4,597 MB, because the peak of most programs comes before they free memory.

Trimming returns less memory than the model subtracts on some programs, because `malloc_trim` can release only whole pages inside free chunks. Perl holds 6.9 MB of free bytes on average over its run, but trimming lowers its time-averaged RSS by 0.3 MB, which means that Perl's free bytes lie in chunks smaller than a page or in pages that also hold live data. On Lua, trimming returns 6.1 of 8.3 MB of free bytes, and on miniVite 4.8 MB, as much as the 4.4 MB of free bytes of its build without the pass. On `cc1`, GCC's garbage-collected heap keeps its own free pages, and libmpr made no trim.

Overhead. The median run time with trimming is 1.01× the glibc run's, and 1.02× under libmpr against 1.00× for libmpr alone. The highest ratios are 1.10×, on SQLite under libmpr with 24 trims per run and on llama.cpp at 32 tokens under glibc, but libmpr without trimming reaches

Table 18: The workloads on which trimming happened (more than one trim in a run): time-averaged RSS (MB), run time over the glibc run's, and the trims per run, mean of 3 runs. On the other 17 workloads, libmpr made at most one trim per run.

	Time-averaged RSS (MB)				Run time (× glibc)		Trims	
Workload	glibc	+trim	libmpr	+trim	+trim	libmpr+trim	glibc	libmpr
miniVite (pass)	23	18	20	19	1.03	1.05	10	7
python	52	48	53	48	1.01	1.01	9	9
lua	45	39	47	39	0.98	1.07	39	38
sqlite	42	41	43	41	1.06	1.10	26	24
perl	52	51	52	52	1.09	1.07	7	6
quicksilver	53	52	52	52	1.01	0.99	20	3
gap-sssp	66	67	67	69	1.00	1.05	4	1
llamacpp-135m-n128	294	291	141	140	1.05	1.05	20	18
llamacpp-135m-n32	203	178	94	89	1.10	1.00	20	18
llamacpp-360m-n64	624	583	335	335	0.97	0.99	20	18
lulesh	96	95	95	93	1.01	1.06	116	96
minife	260	259	260	259	1.03	1.00	12	3

Table 19: Trimming under glibc at three shares s: the trims per run summed over the allocator-heavy programs, the time-averaged RSS summed over the allocator-heavy programs and over all 29 workloads (MB), and the run time over the glibc run's in the same set of runs (×), median and largest over the workloads, mean of 3 runs. The untrimmed row and the row at $s = 0.1$ come from the same runs.

	Trims	Time-averaged RSS (MB)		Run time (× glibc)	
Share s	per run	Alloc.-heavy	All	Median	Largest
none	0	360	3,600	1.00	1.00
0.02	321	342	3,550	1.01	1.09
0.1	111	344	3,524	1.01	1.10
0.3	38	350	3,555	1.00	1.08

1.11× on Perl, so these differences are within the noise of our runs. The RSS integrated over the run does not fall with trimming, 17.1 against 17.3 GB s over all workloads, because the run times vary more than the RSS falls.

Share. We repeated the glibc runs with trimming at $s = 0.02$ and $s = 0.3$ (Table 19). At $s = 0.02$, libmpr trims 2.9 times as often as at $s = 0.1$, 321 against 111 trims per run on the allocator-heavy programs, but the time-averaged RSS of these programs stays at 342 against 344 MB. At $s = 0.3$, it trims 38 times and the RSS falls by half as much, to 350 MB. The memory that trimming returns is therefore limited by the whole free pages that glibc holds and not by how often libmpr checks, and $s = 0.1$ returns nearly all of them. The total over all 29 workloads varies by 30 MB between the sets of runs without a trend in s, because llama.cpp's RSS varies from run to run. The largest run time is 1.09× on Perl at $s = 0.02$, with 40 trims per run.

Returning every free byte. To bound what any allocator could gain by returning its free memory, we also subtracted all of glibc's free bytes from the RSS of the untrimmed runs, which is the RSS that an allocator would hold if it returned every free byte to the kernel at once, as a compacting allocator such as Mesh [23] aims to do. The free bytes come from libmpr's monitor in separate runs without trimming, placed on the same steps of progress. glibc's `mallinfo` counts the free bytes of the main arena only, so the bound misses the free memory of the arenas of other

Table 20: Cluster simulation of the resizing study (DMSched-Sim, 3,000 jobs, the RSS as the job's truth): throughput and mean response time of each policy with glibc's RSS, with trimming at $s = 0.1$ and at the bound of returning every free byte, mean of 3 seeds, and the number of the 6 seed and load pairs on which the throughput is higher than with glibc's RSS. The standard deviation of the response over seeds is about 200 min for the dynamic policies.

		Load 0.8		Load 1.2		Seeds with
Policy	Memory	Jobs/h	Resp. (min)	Jobs/h	Resp. (min)	more jobs/h
Request held	glibc's	21.7	1,844	21.7	2,421	
	trimmed	21.7	1,842	21.7	2,438	4 of 6
Dynamic provisioning	glibc's	21.5	1,149	21.0	1,768	
	trimmed	20.9	1,109	21.4	1,629	1 of 6
	bound	21.7	1,027	21.8	1,615	6 of 6
Dynamic, look-back	glibc's	21.6	1,129	21.8	1,605	
	trimmed	22.4	1,041	22.2	1,557	4 of 6
	bound	22.5	989	23.2	1,459	6 of 6
libmpr	glibc's	25.0	1,246	25.1	1,817	
	trimmed	25.3	1,194	25.4	1,797	6 of 6
	bound	25.7	1,158	25.8	1,755	6 of 6
libmpr + dynamic	glibc's	23.1	775	23.2	1,340	
	trimmed	24.4	705	24.2	1,350	5 of 6
	bound	24.2	682	25.7	1,126	6 of 6
libmpr + dynamic, look-back	glibc's	24.1	741	25.2	1,242	
	trimmed	24.4	708	24.5	1,279	3 of 6
	bound	26.4	559	26.1	1,161	5 of 6

threads, which `malloc_trim` releases. On llama.cpp, the bound is therefore above the trimmed RSS, and we take the smaller of the two at each step. On the simulator's curves, from which the runtime's pages are removed, the time-averaged RSS of the allocator-heavy programs falls from 352 MB under glibc to 335 MB with trimming and to 318 MB at the bound, and their summed peak RSS falls from 537 to 526 and 514 MB. Trimming therefore returns about half of the free memory, and the other half lies in free chunks smaller than a page or in pages that also hold live data, most of it Perl's (51 against 44 MB at the bound).

Cluster. We repeated the cluster simulation of the resizing study with the trimmed builds and with the bound, in which each job is one of the 29 workloads and its demand over time is its RSS (Table 20). A job that holds its request for the whole run gains nothing from trimming, because the request does not change. Under dynamic provisioning with a look-back policy, which resizes the reservation to the RSS of the previous interval, trimming lowers the mean response time at load 0.8 from 1,129 to 1,041 min, and the bound lowers it to 989 min. With libmpr, trimming raises the throughput on every seed and load, but only from 25.0 to 25.3 jobs per hour at load 0.8, and with libmpr and look-back dynamic provisioning, trimming raises it on 3 of 6, and the mean moves from 24.1 to 24.4 at load 0.8 and from 25.2 to 24.5 at load 1.2. The bound raises the throughput of every dynamic policy on every seed and load except one, and with libmpr and look-back dynamic provisioning from 24.1 to 26.4 jobs per hour and the response from 741 to 559 min at load 0.8. The simulation does not charge the run time of trimming, and the bound has no run time, because no allocator we ran achieves it.

We conclude that trimming returns a few megabytes per allocator-heavy program at no mea-

surable cost, but that on this corpus its effect on the cluster is within the variation over seeds, because libmpr's reservations on the large blocks account for most of the memory that the jobs request and do not use. An allocator that returned the free memory in chunks smaller than a page would raise the throughput of a cluster that already resizes the large blocks and the reservations by 0.9 to 2.3 jobs per hour.

13 Hot and cold memory for a memory pool

A system with disaggregated memory gives each node local memory and a shared pool that is slower to reach [12, 22]. The footprint model tells the system how much memory a job holds. To place part of that memory in the pool without slowing the job, the system must also know which memory the job touches rarely. Pin can record every access, but at the cost of Section 5. In this section, we extend the sampling library to measure, at about the cost of a native run, which of a job's memory is hot and which is cold, and we test whether a placement plan built from one profile run lets a job keep a quarter or half of its memory in the pool at a small cost in run time.

13.1 Measuring hot and cold memory without privileges

Accessed bits. The referenced-page alternative of Section 11 clears the accessed bits of the process's pages by writing `1` to `/proc/self/clear_refs`, which needs no privileges. In the new mode (`MPR_MODE=refs`), the library's thread reads `/proc/self/smaps` at the end of each window of W seconds and then clears the bits. For each mapping, `Rss` counts its resident memory and `Referenced` the resident memory touched since the last clear. We call the referenced bytes of a window *hot* and the rest of the resident bytes *cold*. The library writes these per window for the heap, the other anonymous mappings, the stacks, the program's data and code, and for each call site below. Our kernel (4.18) has no DAMON [21], and its idle-page tracking interface (`/sys/kernel/mm/page_idle`) requires root.

Call sites. The kernel reports accessed bits per mapping, so the library makes every large block a mapping of its own. In a build with call sites, it serves each `malloc`, `calloc`, `realloc`, `posix_memalign`, `aligned_alloc` or `memalign` of at least 256 KiB with its own `mmap`, followed by an inaccessible guard page, because the kernel merges adjacent anonymous mappings. It also records the program's own anonymous private mappings of at least 64 KiB, which covers CPython's 256 KiB arenas and GCC's garbage-collected pages. Each block belongs to a site, the stack of up to 8 return addresses that `backtrace` finds above the library's own frames, identified by a hash of each frame's object path and offset. The hash does not depend on where the objects are loaded, so a site has the same identity in every run and on every input. To find the cold parts of a large block, the library also splits each block into 2 MiB chunks. It marks every other chunk with `MADV_DONTDUMP`, which changes no behavior that our programs use, but which keeps the kernel from merging neighboring chunks, so that each chunk is a mapping of its own.

Cost and what is cold. With windows of 0.1 s, the mode costs 1.01× to 1.06× the library's run without a mode, and with call sites 1.03× to 1.21×, the most on miniVite, which allocates many blocks of about 256 KiB (Table 21). Whether memory is cold depends on the window. On the four GAP kernels, a window of 0.02 s references 28% to 45% of the resident anonymous memory, a window of 0.1 s 56% to 69%, and a window of 0.5 s all of it. A window must therefore be short compared with the run, and we use 0.1 s for runs of 1.5 to 4 s and 0.02 s for the smaller profiling inputs below. Call sites cover all of GAP's anonymous memory, 95% of CPython's and 87% of `cc1`'s, but only half of Lua's, 22% of Perl's and none of SQLite's, whose memory lies in blocks

Table 21: Measuring hot and cold memory: run time with windows of 0.1 s over the library's run without a mode, for the zones alone (refs) and with call sites (sites); the share of the resident anonymous memory referenced within a window at three window lengths (memory-time weighted); the share of anonymous memory in site blocks at the peak (site share); and the cold bytes at 2 MiB chunk granularity over the cold bytes at page granularity at the peak, with windows of 0.1 s (cold named). Median of 3 runs, one run for the windows of 0.02 and 0.5 s.

Program	refs	sites	hot 0.02 s	hot 0.1 s	hot 0.5 s	site share	cold named
GAP bfs	1.03	1.03	0.28	0.56	1.00	1.00	0.90
GAP cc	1.03	1.03	0.30	0.57	1.00	1.00	0.90
GAP pr	1.02	1.03	0.45	0.69	1.00	1.00	0.90
GAP sssp	1.05	1.06	0.33	0.57	0.99	1.00	0.76
miniVite	1.01	1.21	0.22	0.24	0.75	0.72	0.12
Quicksilver	1.01	1.09	0.13	0.29	0.55	0.88	0.79
`cc1`	1.06	1.09	0.17	0.35	0.66	0.87	0.77
CPython	1.05	1.09	0.33	0.67	0.97	0.95	0.66
Lua	1.04	1.07	0.57	0.80	0.99	0.50	0.88
Perl	1.05	1.08	0.72	0.86	0.99	0.22	0.64
SQLite	1.03	1.12	0.42	0.91	1.00	0.00	0.00

smaller than 256 KiB. On GAP, the 2 MiB chunks name 76% to 90% of the cold memory that the page-level counts find.

13.2 Placement on an emulated pool

Emulation. Our machine has two NUMA nodes, and we use the second as the pool. Every run binds its threads and its memory to node 0 (`numactl -cpunodebind=0 -membind=0`, from a user installation of numactl), and a plan binds chunks to node 1 with `mbind` and `MPOL_BIND` when the library creates the block, before the program touches it. The firmware's distance table gives 21 for node 1 against 10 for node 0. A pool behind CXL is slower than one NUMA hop [12], so the run times below are a lower bound on the cost of a placement; Section 13.5 models a slower pool. Every placement run turns off transparent huge pages for the process (`MPR_NOTHP`) and records the bytes on each node from `/proc/self/numa_maps` every 0.25 s.

Plans. A profile run with chunks gives, for each chunk, its memory-time, the sum of its resident bytes over the windows, and its referenced memory-time, the sum of its referenced bytes. A plan ranks the chunks by their referenced share, the referenced memory-time over the memory-time, and takes chunks until their memory-time reaches a share f of the memory-time of all anonymous memory. The *cold* plan takes the least referenced chunks first, the *hot* plan the most referenced first, and the *random* plan a random order. All plans with the same f place the same memory-time in the pool, so they differ only in which memory they place. A budget of bytes at the peak would not do this on GAP, whose peak falls in the construction of the graph, before the kernel runs. A chunk is named by its site, the order of its block among the site's blocks, and its position in the block. Each policy runs 5 times, interleaved with the others, and we report the median run time over that of the run with no plan.

Which memory goes to the pool matters. On GAP and CPython, the cold plan keeps the run time within 1% of the run with no plan at $f = 0.25$, while the hot plan with the same memory-time in the pool costs up to 1.117× and all memory on node 1 costs up to 1.160× (GAP pr, Table 22). On GAP, CPython and Lua, the cold plan pays 0 to 17% of the cost of placing all memory in the pool, and the hot plan 32% to 73%. At $f = 0.5$, the cold plan costs at most 1.027×

Table 22: Static placement: run time with a plan over the run with no plan (×), median of 5 runs, for all memory on node 1 (remote) and for plans that place 25% or 50% of the memory-time in the pool, cold, random or hot first. The plans from the smaller input come from GAP at a scale of 16 and CPython and Lua at a quarter of the input, with windows of 0.02 s, applied at a scale of 18 and the full input. They ran in a separate set of runs with its own baseline.

		Plan from the same input					Plan from the smaller input		
Program	remote	cold 25	rand 25	hot 25	cold 50	hot 50	cold 25	hot 25	cold 50
GAP bfs	1.042	1.007	1.007	1.021	1.007	1.035	1.007	1.021	1.014
GAP cc	1.040	1.000	1.007	1.013	1.013	1.027	1.000	1.013	1.007
GAP pr	1.160	1.005	1.058	1.117	1.015	1.150	1.005	1.135	1.010
GAP sssp	1.087	1.005	1.022	1.044	1.022	1.066	1.005	1.049	1.027
miniVite	1.000	1.005	1.000	1.000	1.000	1.000	—	—	—
Quicksilver	1.007	0.998	1.000	1.002	1.000	1.005	—	—	—
`cc1`	1.016	1.000	1.005	1.005	1.005	1.011	—	—	—
CPython	1.048	1.008	1.019	1.021	1.027	1.045	1.008	1.037	1.013
Lua	1.151	1.023	1.038	1.049	1.053	1.060	1.049	1.042	1.049
Perl	1.200	1.028	1.028	1.028	1.028	1.028	—	—	—

on GAP and CPython. The memory that the cold plan places is not idle: on GAP, 31% to 34% of its memory-time is referenced within a window, and the remote node serves these accesses at a small cost. The random plan lies between the two. miniVite, Quicksilver and `cc1` run at most 1.016× slower with all memory in the pool, so their placement does not matter. On Perl, the plans can place only the 17% of the memory-time that call sites cover, and all three plans give the same chunks.

Plans from a smaller input. A plan named by the block's order and the chunk's position does not carry over to a larger input, whose blocks are larger and more numerous. A *relative* plan instead names, for each site, the ranges of the block, in 16 equal parts, to place, pooled over all the site's blocks. We profiled GAP at a scale of 16 and CPython and Lua at a quarter of the input, and applied the relative plans at a scale of 18 and at the full input. On the larger input, 90% to 100% of the site memory-time comes from sites that the smaller run saw. The transferred cold plan matches the plan from the same input on GAP and CPython, 1.000× to 1.008× at $f = 0.25$, and the transferred hot plan still costs 1.135× on GAP pr. The memory-time that the transferred plans place differs from f by up to 0.11 on CPython, where the cold plan at $f = 0.25$ places 14% of the memory-time and the hot plan 31%. On Lua, the plans cover the same few sites, and all three cost about 1.045×.

13.3 Moving memory while the program runs

A plan needs a profile run. A runtime could instead move memory between the nodes while the program runs, by scoring each chunk from the accessed bits of the past windows and moving chunks with `mbind` and `MPOL_MF_MOVE`, as tiered-memory systems do [15, 24, 11]. We tested three policies in the library (`MPR_MIGRATE`). Each window, a chunk's score is α times its referenced share in the window plus $1 - \alpha$ times its previous score. The *fill* policy ($\alpha = 0.5$, windows of 0.1 s) keeps the lowest-scored chunks in the pool up to f of the anonymous RSS. The *threshold* policy moves only chunks scored at most 0.1. The *slow* policy uses $\alpha = 0.1$ and windows of 0.25 s, so that a chunk must stay cold for several windows before it moves.

None of the three policies approaches the static plan (Table 23). Monitoring alone costs 1.013×

Table 23: Moving memory while the program runs, against the static cold plan from a profile of the same input: run time over the run with no plan (×), median of 5 runs. Monitor: the accessed bits sampled and cleared at the window length, no memory moved. Fill, threshold and slow at $f = 0.25$ or 0.5 of the anonymous RSS. Three sets of runs, each with its own baseline.

Program	remote	static cold 25	monitor 0.1 s	fill 25	threshold 25	monitor 0.25 s	slow 25	slow 50
GAP bfs	1.042	1.007	1.021	1.083	1.021	1.014	1.034	1.048
GAP cc	1.034	1.000	1.013	1.074	1.020	1.013	1.034	1.047
GAP pr	1.155	1.000	1.019	1.130	1.024	1.010	1.058	1.106
GAP sssp	1.088	1.011	1.055	1.148	1.049	1.022	1.060	1.087
CPython	1.037	1.008	1.029	1.077	1.036	1.015	1.037	1.075
Lua	1.151	1.026	1.038	1.125	1.038	1.019	1.053	1.083

to 1.055× with windows of 0.1 s and 1.010× to 1.022× with windows of 0.25 s, because each sample reads every mapping and clears every accessed bit. The fill policy costs 1.074× to 1.148× at $f = 0.25$, more than placing all memory in the pool on four of the six programs. In the steady state of these programs, too little memory stays cold for a window to fill the budget, so the policy places hot memory and moves it back and forth. The threshold policy finds less than 3% of the memory-time cold enough to move, so its cost is that of monitoring. The slow policy moves 68 to 205 MB per run and costs 1.034× to 1.060× at $f = 0.25$, against 1.000× to 1.026× for the static plan, because it judges new chunks before their accesses begin. The static plan places memory before the first touch, so it moves nothing, it ranks chunks over the whole run, and it needs no monitoring. On runs of a few seconds, the profile run is therefore worth more than run-time migration. A program whose phases last much longer than a window may favor migration, but we did not test one.

13.4 Placement in libmpr

Mechanism. libmpr already serves each large block of a site with a reservation from a mapping of its own, so it can bind ranges of the block to the pool before the program touches them. We added placement lines to its reservation format, `key place K lo`$_1$ `hi`$_1$ `...`, which bind K ranges of each block of the site, given as shares of the block, to the pool node (`MPR_PLACE`, `MPR_NODE`). A line is independent of the site's reservation, and a site with a placement line and no reservation is served and committed in full. libmpr also intercepts `mmap` and places the program's own anonymous mappings by their caller's line. libmpr names a site by its return address in its object file, with up to two frames above it as a context, so a converter (`place_lplans.py`) maps each profiled stack to libmpr's key, skips the frames of `operator new` in the C++ runtime, and pools the stacks that share a key. GAP's stacks map to keys one to one, and CPython's 62 stacks and Lua's 8 map to 31 and 6 keys. Finding a site's context takes a `backtrace` and two `dladdr` lookups, which on CPython, with about 2,700 arena mappings per run, cost 1.04× even with all memory left on node 0. libmpr now caches the site of each context by its frame addresses, which brings this cost to 1.01×.

Results. Through libmpr, the transferred cold plan costs 1.005× to 1.007× on GAP and 1.017× on CPython at $f = 0.25$, against 1.136× for the hot plan on GAP pr, which matches the library's own placement (Table 24). libmpr with no plan costs 1.00× to 1.02× the run without it. On CPython, the random plan at $f = 0.25$ and the hot plan at $f = 0.5$ cost more than placing all memory in the pool (1.061× and 1.072× against 1.049×). These plans bind sixteenths of the 256 KiB arenas, which splits each arena into mappings of 16 KiB, and we suspect that the kernel's

Table 24: Placement through libmpr, plans from the smaller input except "same 25": libmpr with no plan over the run without libmpr (libmpr), and run time with a plan over libmpr with no plan (×), median of 5 runs. Pool share: the share of the anonymous memory-time on node 1 under the transferred cold plan at $f = 0.25$.

Program	libmpr	remote	same 25	cold 25	rand 25	hot 25	cold 50	hot 50	pool share
GAP bfs	1.000	1.042	1.007	1.007	1.021	1.028	1.014	1.035	0.30
GAP cc	1.007	1.041	1.007	1.007	1.014	1.020	1.014	1.034	0.30
GAP pr	1.010	1.155	1.005	1.005	1.044	1.136	1.015	1.150	0.23
GAP sssp	1.006	1.088	1.005	1.005	1.027	1.055	1.022	1.066	0.25
CPython	1.012	1.049	1.020	1.017	1.061	1.049	1.029	1.072	0.12
Lua	1.020	1.159	1.050	1.050	1.050	1.062	1.066	1.070	0.25

Table 25: Reservations and placement together through libmpr: peak commit, peak anonymous memory on node 0 and on node 1 (MB), and run time over libmpr with neither (×), median of 5 runs. Plans profiled on the same input. Every run turns transparent huge pages off.

Program	Configuration	Commit	Node 0	Node 1	Time
darknet	libmpr alone	511	271	0	1.000
	reservations	272	271	0	1.003
	cold 25	511	201	70	1.009
	reservations + cold 25	272	201	70	1.008
	reservations + cold 50	272	147	124	1.016
	reservations + hot 50	271	125	146	1.017
	remote	511	0	271	1.024
	reservations + remote	272	0	271	1.030
llama.cpp	libmpr alone	354	202	0	1.000
	reservations	22	22	0	0.832
	cold 25	355	14	8	0.836
	reservations + cold 25	24	14	8	0.832
	reservations + cold 50	24	14	8	0.834
	reservations + hot 50	22	19	3	0.829
	remote	354	0	202	1.072
	reservations + remote	22	0	22	0.854

handling of these small mappings costs more than the remote accesses, but we did not confirm it.

Reservations and placement together. The reservations decide which memory is backed, and the plan decides on which node the backed memory lies. We ran both on darknet and llama.cpp with the 135M model generating 32 tokens, the two workloads on which the reservations of the resizing study save the most, with plans profiled on the same input (Table 25). All 110 runs gave the output of the glibc run.

On darknet, the two add up. The reservations lower the peak commit from 511 to 272 MB, because darknet never touches some of the blocks it allocates, and the cold plan at $f = 0.5$ then moves 124 MB of the memory it touches to the pool, which lowers the local memory from 271 to 147 MB at 1.016×. darknet runs only 1.03× slower with all memory in the pool, so the hot plan costs about as little.

On llama.cpp, the reservations alone lower the anonymous memory from 202 to 22 MB, because libmpr does not zero the pages of the key-value cache that the program never touches, and they make the run 0.83× as long. The rest of its resident memory is the model's weights, which the engine

maps from its file and which no plan places. The first profile covered only 6.5% of llama.cpp's memory-time, because the library did not intercept the aligned allocations through which ggml allocates its buffers. With `posix_memalign`, `aligned_alloc` and `memalign` intercepted, the plans cover 27% and 54%. The cold plans then place 8 MB in the pool and leave 14 MB on node 0, at no measurable cost, and the hot plan places 3 MB. The plans place far less than f because the profile ran under glibc, which zeroes the whole key-value cache and so makes it resident and cold, whereas libmpr never backs most of it. A plan meant to run with reservations should therefore be profiled with them. The placement lines also add 1.8 MB of commit, because libmpr serves the sites they name in full, where it would otherwise leave them to glibc. Without reservations, the cold plan alone gives most of the reservations' effect for the same reason: libmpr serves the plan's sites, including the key-value cache, and does not zero their untouched pages.

13.5 A slower pool

Node 1 is a pool one NUMA hop away: a pointer chase over 512 MB takes 101 ns per load on node 0 and 157 ns on node 1, so the pool adds 56 ns. Memory behind CXL adds more than a NUMA hop [12], and our machine offers no way for an unprivileged user to slow node 1 down (throttling the uncore clock needs the model-specific registers, and Broadwell has no memory bandwidth allocation). We therefore model a slower pool from the runs we can make.

Model. For each program, the time a placement adds is the number of the program's reads that the pool serves times the pool's added latency d times a constant s, the inverse of the program's memory-level parallelism: $T(d) = T(56\,\text{ns}) + s\,N\,(d - 56\,\text{ns})$. We count N with `perf stat` in user mode, as the reads (loads, stores' reads for ownership and prefetches) that missed the last-level cache and were served by node 1's memory, and fit s by least squares over the runs whose policies differ only in where the memory lies: for GAP, CPython and Lua every plan against libmpr with no plan, since libmpr serves the same blocks under both; for darknet and llama.cpp only all memory local against all memory in the pool, with and without the reservations, since there a plan also makes libmpr serve the sites it names and skip zeroing their untouched pages. The same runs, five of each, use the plans, libmpr and settings of Table 24 and Table 25.

Check. To test the model at a second latency, we ran every configuration again with eight threads on node 1's CPUs streaming over node 1's memory. The pointer chase then takes 209 ns on node 1, so the pool adds 107 ns, and 102.5 ns on node 0. Predicted from the runs at 56 ns alone, the slowdowns at 107 ns are within 0.015 of the measured ones at the median and within 0.02 to 0.07 for every program but GAP bfs (0.11), whose run without a plan itself slows by 5% beside the streaming threads (Table 26). The model errs high: the prediction exceeds the measurement in 74% of the configurations, by 0.013 on average, so the slowdowns it gives at higher latencies are, if anything, too large. The streaming threads also take bandwidth from the pool, which the check therefore includes, but a CXL link with less bandwidth than a socket's memory controllers could cost bandwidth-bound programs more than the model says.

The cold plan stays cheap on a slower pool. At 300 ns, about 4× the local latency, placing all anonymous memory in the pool costs 1.38× to 1.94× on GAP, 1.46× on CPython and 2.31× on Lua, and the hot plan at $f = 0.25$ costs 1.17× to 1.71× on GAP. The transferred cold plan still costs 1.03× to 1.07× on GAP and CPython, 4 to 11% of the all-pool cost on GAP. The model's cost is linear in the latency, so the slower pool widens the gap between the cold and the hot plan without changing their order. On Lua the cold plan costs 1.26× at 300 ns; it already paid 35% of the all-pool cost at 56 ns, the most of any program. With their reservations, darknet's cold plan, which places 70 MB, costs 1.04×, and llama.cpp's, which places 8 MB, 1.09×.

The pool in the cluster. The cluster simulation of Section 12 charges a job that holds a

Table 26: A slower pool: run time with all anonymous memory in the pool or with the transferred cold or hot plan at $f = 0.25$ over the run with all memory local (×, median of 5 runs), for the pool's added latency over local memory of 56 ns (node 1 idle) and 107 ns (node 1 busy), measured, and of 200 and 300 ns, about 3× and 4× the local latency, predicted. darknet and llama.cpp run with their reservations, over the reservations alone. Error: the largest difference between the predicted and the measured slowdown at 107 ns.

		All in the pool				Cold 25				Hot 25			
Program	Error	+56	+107	+200	+300	+56	+107	+200	+300	+56	+107	+200	+300
GAP bfs	0.112	1.092	1.068	1.345	1.522	1.013	1.000	1.031	1.043	1.065	1.050	1.194	1.283
GAP cc	0.023	1.075	1.116	1.256	1.382	1.000	1.018	1.017	1.030	1.025	1.055	1.113	1.174
GAP pr	0.023	1.176	1.311	1.628	1.943	1.009	1.000	1.025	1.036	1.140	1.267	1.477	1.711
GAP sssp	0.021	1.112	1.201	1.415	1.626	1.000	1.010	1.041	1.070	1.061	1.116	1.195	1.288
CPython	0.041	1.060	1.102	1.297	1.462	1.016	1.024	1.048	1.070	1.066	1.075	1.184	1.266
Lua	0.063	1.218	1.382	1.864	2.314	1.076	1.095	1.184	1.259	1.080	1.125	1.209	1.298
darknet	0.020	1.038	1.044	1.112	1.163	1.010	1.014	1.029	1.041	1.014	1.031	1.040	1.057
llama.cpp	0.071	1.050	1.065	1.182	1.274	1.033	1.016	1.066	1.088	1.000	1.000	1.042	1.072

share f of its memory-time in the pool a run time of $1 + \beta f$, with $\beta = 0.5$ for every job. We replaced this cost with one measured per workload: we ran each of the 29 workloads under libmpr with all memory local, all memory on node 1, and the random plan at $f = 0.25$ and the cold and hot plans at $f = 0.25$ and 0.5, each made from the workload's own profile, under `perf stat` as above (3 runs each, node 1 idle and busy). The model above gives each policy's run time at an added latency d, and a job's cost at f is interpolated between the measured plans, from no cost at $f = 0$ to all memory in the pool at $f = 1$. Only the reads that a plan adds over the local run count, because node 1 also holds the page cache of files that a local run maps, such as llama2.c's weights. Fitted on the idle node, the model predicts the slowdowns on the busy node within 0.008 at the median. It errs most on HPCCG and NPB CG (0.17 and 0.24), which stream through memory and slow down beside the streaming threads more than the pointer chase does. Each job also reads the pool at its workload's measured rate per thread times its cores, so the cluster's demand on the pool follows from the schedule. A pool of bandwidth B then has the latency that node 1 measured at the same share of its own bandwidth (53 GB/s at most; the pointer chase takes 149 ns idle and 205 ns at saturation), and above B the pool's part of each run is stretched by the excess. We run the simulation at the idle latency, take the mean demand over the steady part of the run, set the loaded latency from it, and run again, halving the step, until the latency moves by less than 1 ns; 1,285 of the 1,296 simulations with a bounded bandwidth settle in 10 runs or fewer, and the other 11 alternate between two latencies 2 ns apart.

The flat cost hides the placement. With the measured costs, every policy runs more jobs per hour than with $\beta = 0.5$ (Table 27): at 56 ns and with the cold plan, libmpr runs 30.0 against 25.1 jobs per hour and the request held 25.9 against 21.6. The pool holds about 30% of the jobs' memory-time, and the mean stretch of a job is 1.06 under the flat cost and 1.017 under the cold plan at 56 ns. At 300 ns, libmpr's throughput falls to 26.4 jobs per hour with the cold plan and to 25.4 with the random or the hot plan, close to the flat cost, and the mean stretch is 1.043 with the cold plan, 1.052 with the random and 1.054 with the hot one. libmpr's lead over the request held stays at 3.0 to 4.1 jobs per hour, 13 to 16%, at every latency and placement, so a slower pool does not change the order of the policies. The bandwidth matters little at this scale: the jobs read the pool at 12 GB/s on average and at 21 GB/s at most, so a pool of 32 GB/s adds 13 to 17 ns to its latency on average (42 ns at most) and costs 0.24 jobs per hour on average against a pool of unbounded bandwidth, less than the variation over seeds. The mean demand, however, smooths over the minutes in which several jobs that stream through memory share the pool.

Table 27: Cluster simulation with a slower pool (the setup of Table 20, load 0.8, 32 nodes of 128 GB and a 1,024 GB pool of 64 GB/s): throughput in jobs per hour, mean of 3 seeds, with the flat cost $\beta = 0.5$ and with each workload's modelled cost at the pool's idle added latency over local memory in ns, when the pool holds the job's coldest, a random or its hottest memory. The standard deviation over seeds is about 0.8 jobs per hour.

		Cold first				Random	Hot first
Policy	$\beta = 0.5$	+56	+107	+200	+300	+300	+300
Request	21.6	25.9	25.5	24.1	22.8	22.3	22.4
libmpr	25.1	30.0	29.0	27.7	26.4	25.4	25.4
Dynamic provisioning	21.2	25.5	24.9	23.7	21.8	21.7	21.7
libmpr + dynamic	23.6	28.8	27.0	26.1	25.0	24.2	23.9
Pond	21.1	25.2	24.6	23.6	22.5	21.8	21.8
libmpr + Pond	25.3	30.1	29.1	27.7	26.3	25.5	25.4

Table 28: The pool minute by minute, at an idle added latency of 300 ns: the jobs' demand on the pool (mean, 95th percentile and largest minute), the share of the steady minutes in which it exceeds the bandwidth, the pool's added latency, and the throughput with each placement, means over the 6 policies, both loads and 3 seeds (36 runs per cell); the last column is the throughput of the cold plan with the latency set from the mean demand (Table 27), which the 16 GB/s pool was not run with.

	Demand (GB/s)			Minutes	Latency (ns)		Jobs/h, minute by minute			Jobs/h,
Bandwidth	Mean	p95	Max	over	Mean	p95	Cold	Random	Hot	averaged
256 GB/s	12.5	28.3	39.9	0%	302	306	24.6	23.7	23.8	24.3
64 GB/s	12.5	28.2	40.2	0%	312	331	24.9	23.8	23.7	24.1
32 GB/s	12.2	26.7	38.3	2%	331	393	24.4	23.6	23.5	24.0
16 GB/s	12.7	27.3	37.8	30%	377	447	23.6	22.5	22.3	—

Minute by minute. To see the peaks, we made the simulator set the pool's latency every minute. Each minute, the jobs that run read the pool at their rates at the current latency; the sum gives the pool's latency and stretch as above, and each job advances by $1/(1 + e)$ minutes of its run with all memory local, where e is its cost at that latency. The scheduler commits a job's resources for the run time at the latency of the minute it starts; a job that finishes early releases the rest, and one that runs longer keeps its last minute's resources one minute at a time. In the 1,728 simulations, no extended job ever needed more cores than its node had. The pool's demand is bursty (Table 28): at 300 ns it averages 12.5 GB/s, but its 95th percentile is 27 to 28 GB/s and its largest minute 38 to 40 GB/s, so a pool of 32 GB/s is saturated in 2% of the minutes and one of 16 GB/s in 30%, where the latency reaches that of node 1 at saturation. Where the pool is not saturated, the minute-by-minute throughput agrees with the averaged one: over the configurations the two have in common, the difference is +0.1 to +0.3 jobs per hour on average, below the standard deviation over seeds of 0.85. At 16 GB/s, the throughput falls by 1.0 jobs per hour with the cold plan against a pool of 256 GB/s and by 1.2 and 1.5 with the random and the hot plan, because the cold plan reads the pool least: at 16 GB/s, the mean stretch of a job is 1.058 with the cold plan and 1.077 with the hot one. The bandwidth therefore matters only when the pool's bandwidth is near the jobs' peak demand, and then a cold placement matters more, not less.

A correction to the resizing runs. While testing these changes, we found that libmpr read each reservation line into a buffer of 512 bytes and dropped the rest of a longer line. The only lines this long are those of llama.cpp's key-value cache, which list 60 to 120 ranges, of which

libmpr kept the first 24. libmpr now reads lines of up to 8,192 bytes and skips a longer line with a warning instead of reading part of it. With the whole lines, llama.cpp commits 0.7 MB more at 32 tokens with the 135M model and 1.9 MB more at 64 tokens with the 360M model, and its RSS and output do not change. On the other 23 programs, the new libmpr gives the same per-site commit, the same output and a peak commit 8 KB higher than before. We ran the resizing study's llama.cpp configurations again with the corrected libmpr (own, held-out and pooled reservations, unseen inputs, the build with the pass, thread stacks and native profiles; 351 runs, every output identical): the commit under libmpr rose by 0.4 to 2.7 MB with each model's own profile and by up to 7.3 MB with held-out and pooled ones, whose lines are longer, and its commit stays 77 to 90% below glibc's. The runs of Section 12 used the truncated lines too; they measure RSS, which the correction does not change, and we keep them.

14 Costs and effects of the approaches

The approaches that we measured fall into four groups, which differ in what they change and in when they cost (Table 29). The first group estimates the footprint for a scheduler and leaves the job's memory unchanged. Its cost is paid once per program and input, and a production run costs nothing. The second group lowers the memory that the job holds, by returning free memory to the kernel or by committing only the predicted part of each large block. Its cost is paid on every run, and libmpr's reservations also need a profile. The third group changes the scheduler, which reserves less than the request and resizes the reservation while the job runs, at the cost of resizes and of restarts when a growth does not fit. The fourth group, placement, leaves the job's memory and the scheduler's reservation unchanged and decides which part of the memory lies in a slower pool.

Estimating the footprint. The data estimate gives MemPrint's accuracy at the cost of a native run, and the other cheap estimates are either less accurate (RSS, the trim alternatives) or below the footprint (referenced pages), which a scheduler cannot use without a margin. Pin remains the only route to a per-site profile, and MemPrint costs 12.0× at the median.

Lowering the memory. Each approach in this group acts on a different part of the waste. libmpr's reservations act on the large blocks that the program requests and does not touch, which is where most of the request's waste lies on this corpus: they raise the throughput from 21.7 to 25.0 jobs per hour without changing RSS. Trimming acts on the small blocks that the program has freed. From `free`, it costs 1.01× at the median and returns half of the free memory, and the other half needs an allocator that can return memory in chunks smaller than a page. The cost of such an allocator is the open question of this group. Mesh reports its own costs, and we did not run it.

Placing memory in a pool. A static plan from one profile run places a quarter of the memory-time in the pool at 1.00× to 1.01× on GAP and CPython, and it combines with libmpr's reservations: on darknet, the reservations lower the commit by 47% and the plan then lowers the local memory by 46%, at 1.016×. Moving memory while the program runs costs more than the static plan on every program we ran, and the monitoring alone costs up to 1.06×.

Reserving less. Dynamic provisioning gains the most in response time, 1,844 to 1,129 min, and its costs are those of the cluster, resizes and restarts, and not of the job. The approaches combine. libmpr with look-back dynamic provisioning gives 741 min, and the bound on returning free memory lowers it to 559 min, which is below the 630 min of the exact curve without libmpr. The exact curve with libmpr, 340 min, is the limit that a predicted curve would approach.

Table 29: The approaches to the gap between RSS and the footprint that we measured, in this paper and in the resizing study, grouped by where they act. One-time cost: paid once per program and input or model, as a run time over the native run's (×). Cost per run: the run time of every production run over glibc's (×), median and the largest we observed. Effect: on the 7 allocator-heavy programs unless stated (whole-run error against the truth, or memory), and in the cluster simulation at load 0.8 (jobs per hour and mean response time).

Approach	One-time cost	Cost per run	Effect	Limit
Estimate the footprint for the scheduler (the job's memory does not change)				
RSS of a native run	1.00×	none	+16% whole-run error	counts free memory and libraries
MemPrint W5 (Pin)	12.0×, 18.7× to 93.0×	none	−5%; per-site attribution	Pin's cost on allocator-heavy programs
Data estimate (Section 3)	0.99×, 1.21× darknet	none	−4%; −4% transferred; 433 against 515 min for the RSS curve (Section 7)	glibc, jemalloc or TCMalloc only; THP; runs of a few ms
Data estimate, `smaps` every 1 ms (Section 6)	1.03× the program alone, 1.25× darknet	none	transferred: median |error| 2.1% against 2.8% (all 23), NPB-MG −50% to +1%	atax −33%; faster `status` counters lag
Data estimate, B from the same run (Section 10)	0.99×	none	+0.6% (all 23); 2.0% on the blind programs	miniVite +30% (its libraries)
Prediction from source, no run (Section 10)	interpreter, 1× to 1,340×	none	peak within 1.1% on PolyBench, +45% on GAP, +57% LULESH	array programs it can run; counts freed bytes
Borrowed growth (Section 10)	one smaller run	none	no better than $\beta = 1$	
Runtime statistics	0.96×	none	+5% CPython, −19% Lua	two programs
Referenced pages	1.02×	none	−24%, −70% miniVite	below the footprint: unsafe
Lower the memory the job holds (the runtime changes)				
glibc trim threshold 0	none	1.02×	RSS error +17% to +14%	top of the heap only
`malloc_trim`, thread every 5 ms	none	1.03×, 1.23× darknet	RSS error +17% to +11%	second thread slows single-threaded programs
`malloc_trim` from `free` (Section 12)	none	1.01×, 1.10×	time-averaged RSS −4.4%, peak unchanged; jobs/h +0.3 with libmpr	whole free pages only
Every free byte returned (bound)	none	not measured	time-averaged RSS −9.7%, peak −4.3%; jobs/h +0.7 with libmpr, +2.3 with libmpr and look-back	needs an allocator that compacts
libmpr reservations	Pin 217× median, or libmpr's profile, one native run	0.92× to 1.03×	23 programs: peak commit −10.7%, RSS unchanged; 29 workloads: jobs/h 21.7 to 25.0	blocks of 64 KiB or more
libmpr `MAP_NORESERVE` stacks	none	0.87× to 1.13×	commit −8 to −34 MB per multithreaded program	commit only
Place cold memory in a pool (the runtime changes where memory lies, Section 13)				
Accessed bits per site and chunk	profile run, 1.03× to 1.21×	none	names 76% to 90% of GAP's cold memory	blocks of 256 KiB or more
Static cold plan, mbind at first touch	one profile run, smaller input works	1.00× to 1.03×	25% of the memory-time in the pool at 0 to 17% of the all-remote cost; darknet local memory 271 to 147 MB with reservations	one NUMA hop; model: up to 1.07× at 300 ns
Migration between windows	none	1.02× to 1.15×	worse than the static plan	monitoring costs 1.01× to 1.06×
Reserve less than the request (the scheduler changes)				
Dynamic provisioning, look-back	none	resizes and restarts, not charged here	response 1,844 to 1,129 min	a job restarts when its growth does not fit
libmpr and look-back	as libmpr	as both	response 741 min; 559 min at the bound	
Exact curve (upper bound)	—	—	30.5 jobs/h, 630 min; 34.3 and 340 min with libmpr	needs the future

15 Limitations

A limitation of the free-byte correction is that it reads only the statistics of glibc, jemalloc and TCMalloc, which leaves the memory of other allocators in the estimate. The arenas of CPython's object allocator and GCC's garbage-collected heap stay in the estimate as RSS. On our corpus, the data estimate is nevertheless within 5% of the truth on CPython and `cc1`. Routing CPython's objects through glibc (`PYTHONMALLOC=malloc`) does not make the estimate more accurate, and it changes the program. Under this setting, the data estimate's whole-run error is −2.5% against a truth traced under the same setting, against −2.9% without it, while the truth's peak rises by 15%, the run's peak RSS by 7%, and its run time by 1.56×. On `cc1`, we counted GCC's own mappings by interposing `mmap` and `munmap` in the sampling library. They hold 99 MB of the 114 MB of anonymous memory at the peak, where glibc's free bytes are 2.8 MB. GCC's statistics at the end of the compilation (`-fmem-report`) show 12 MB of the 94 MB of its garbage-collected heap unused (13%), which the data estimate counts as data. It is unclear whether the 5% result holds for inputs on which these allocators retain more memory.

A limitation of the data-file rule is that it classifies mappings by path, which counts any mapped file that is not a shared object as data. A program that maps a large file it uses as code, or a library without `.so` in its name, would have that file counted.

A limitation of the free-byte correction is also that some free bytes may not be resident, such as the untouched part of the top of the heap, which makes the data estimate lower than the truth. The data estimate is below the truth on 6 of the 7 allocator-heavy programs under glibc, by 1% to 9% over the run, and on SQLite under TCMalloc by 22%, so a scheduler that uses it needs a margin.

A limitation of the sampling thread is that it adds a second thread to a single-threaded program, which slows darknet by 1.21× (Section 5). Sampling from an external process avoids the thread, but the allocator's statistics can be read only inside the process. libmpr's trimming avoids the thread by checking from `free` (Section 12), but a monitor cannot, because it must sample while the program allocates without freeing. A second limitation of the thread is its 5 ms interval, which misses the curve of runs shorter than a few tens of milliseconds and limits the model's transfer from the smallest inputs (Section 6 and Section 7). Sampling `smaps_rollup` every 1 ms at the same cost corrects NPB-MG's transfer, but atax's transfer stays −33%, and the kernel's cheaper RSS counters lag the page tables on runs of a few milliseconds (Table 5).

A limitation of the placement study is its pool, a second NUMA node one hop away, which is faster than a pool behind CXL. We model a slower pool from the counts of the reads the pool serves and check the model at a second, measured latency, but the model is linear in the latency, misses part of the cost to programs that stream through memory (HPCCG and NPB CG, by up to 0.24), and runs on a node with CPUs, where the kernel's placement of its own memory differs from that on a CPU-less pool. Its runs last 1.5 to 4 s, which favors a static plan over migration, and its plans place only blocks of 256 KiB or more and the program's own mappings, which leaves SQLite and most of Perl and Lua unplaced. Node 1 had about 400 MB free, which limited the inputs. The cluster simulation with the slower pool scales each job's cost and pool traffic from runs of its workload and takes them as constant over the job's run, so the peaks of demand within a job are not modelled; the pool's latency follows only the mix of jobs that run.

The evaluation covers a single machine, short runs of 0.1 to 12 s and single-process programs. The truth's clock mapping passes through the native run's reference counts, which also place MemPrint's curve, so MemPrint's timing accuracy is overstated against the truth. The truths under jemalloc and TCMalloc use the clock of the native run under glibc.

16 Related work

Footprint measurement. Valgrind's Massif and DHAT [18, 17] and MemPrint measure the heap or the touched memory of every allocation by instrumenting the program. Our model reads counters that the kernel and the allocator already keep. Unlike MemPrint, it does not attribute memory to allocation sites or show which bytes of a block the program touches.

Page access sampling. DAMON [21] and Thermostat [1] sample the accessed bits of pages to find cold memory, and PEBS sampling [19] records the addresses of sampled loads. These methods measure which pages a program uses within a period of time, which is the working set [4] and not the footprint. Our referenced-page alternative (Section 11) shows that the working set of a window falls far below the footprint on allocator-heavy programs.

Tiered memory. TPP [15], HeMem [24] and Memtis [11] move pages between fast and slow memory while the program runs, from sampled accesses or the kernel's page faults, and Pond [12] sizes a virtual machine's local memory and places the rest in a CXL pool. Our placement decides per call site before the program runs, from one profile run, and our runs show that on short jobs this beats moving memory from the accessed bits during the run.

Memory prediction for scheduling. KS+ [2] predicts a workflow task's memory over time from the RSS of earlier runs, and Zacarias et al. [25, 26] predict the memory of HPC jobs from earlier runs. Pond [12] predicts how much of a virtual machine's memory goes untouched. These systems learn from RSS or from allocated memory, and our model gives them a training signal without the allocator's retained memory and the job's shared libraries at the same cost as RSS.

Allocators and huge pages. Mesh [23] compacts the heap to return fragmented memory to the kernel, and TCMalloc's Temeraire [8] releases free huge pages. Ingens [10] and HawkEye [20] limit the memory that huge pages waste on partly used ranges. These systems reduce the gap between RSS and the footprint by changing the system, whereas our model measures the gap without changing the program's allocations.

17 Conclusion

RSS overestimates the footprint of programs that allocate and free memory throughout their run, because the allocator keeps freed memory resident and because RSS counts shared libraries. A model that subtracts the allocator's free bytes from the anonymous RSS and adds mapped data files and the runtime's pages reduces the median whole-run error on these programs from +16% to −4%, which matches MemPrint's accuracy on the job's own input and on larger inputs, at a median of 0.99× the run time of a plain RSS run instead of 34.4×. In a scheduling simulation, its curve gives the lowest response time of the estimates that profile the job's own input. Pin-based profiling remains needed for attributing memory to code, for allocators the model cannot query, for the curves of runs too short to sample, and for the 4 KiB footprint of programs whose memory the kernel backs with huge pages. Used at run time, the same statistics let a runtime return the allocator's free memory to the kernel, which lowers the RSS of allocator-heavy programs by a few percent at no measurable cost, but which does not change the throughput of a cluster that already resizes the large allocations. glibc returns only whole free pages, which is half of the free memory, and an allocator that returned the rest would raise that cluster's throughput by 0.9 to 2.3 jobs per hour. The model needs no training on the program: it takes the runtime's pages from the job's own run, it measures the companion study's blind programs within a median of 2.0%, and for a program that cannot be run before it is scheduled, the interpreter's footprint plus the runtime's pages of other programs predicts array programs within 1.1% of the peak, but programs that free

memory need the model's own run. Static features of the code and the executable relate to the model's terms, and four code embeddings, up to 8B parameters, do not relate to how the footprint grows with the input. The same library measures which memory is cold per call site at close to the cost of a native run, and a plan from one profile run, carried to a larger input and applied by libmpr with its reservations, keeps a quarter of a job's memory-time in a slower pool at a cost of at most 1% on GAP and CPython, and a latency model predicts at most 7% on a pool four times slower than local memory. In a cluster simulation with these costs, the pool's latency, not its bandwidth, sets the throughput, and placing the cold memory in the pool keeps one more job per hour than a random placement at 300 ns.

Artifacts. `paper_rss/tools/mprshim.c` is the sampling library. `run_model.sh`, `run_transfer.sh`, `run_options.sh`, `run_alloc.sh` and `run_llm.sh` run the experiments, `grid.py` builds the simulator's corpus and runs the scheduling grid, and `compute.py` produces every table. The runs are in `~/memprint-data/2026-10-26-rsscurve`. For Section 13, `run_refs.sh`, `run_place.sh`, `run_lplace.sh` and `run_combo.sh` run the experiments, `place_plans.py`, `place_rplans.py` and `place_lplans.py` build the plans, `regress_libmpr.sh` compares two builds of libmpr, and `place_tables.py` produces the tables, from `~/memprint-data/2026-11-07-refs` and `2026-11-08-lplace`. For Section 13.5, `run_cxl.sh` (with `cxl/hog.c` and `cxl/chase.c`) runs the counted runs at both latencies and `cxl_model.py` fits the model, from `2026-11-08-cxl`. `run_cxlsim.sh` profiles and runs the 29 workloads for the cluster and measures node 1's loaded latency, and `cxl_sim.py` runs the cluster simulation, averaged and minute by minute, from `2026-11-08-cxlsim`.

References

[1] Neha Agarwal and Thomas F. Wenisch. Thermostat: Application-transparent page management for two-tiered main memory. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 631–644, 2017.

[2] Jonathan Bader, Ansgar Lößer, Lauritz Thamsen, Björn Scheuermann, and Odej Kao. KS+: Predicting workflow task memory usage over time. In *IEEE 20th International Conference on e-Science*, 2024.

[3] Scott Beamer, Krste Asanović, and David Patterson. The GAP benchmark suite. *arXiv preprint arXiv:1508.03619*, 2015.

[4] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[5] Jason Evans. A scalable concurrent malloc(3) implementation for FreeBSD. In *BSDCan*, 2006.

[6] Free Software Foundation. *The GNU C Library Reference Manual*, 2024. Section "Memory Allocation Tunables" (`M_MMAP_THRESHOLD`), `https://www.gnu.org/software/libc/manual/`.

[7] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, and Assefaw H. Gebremedhin. miniVite: A graph analytics benchmarking tool for massively parallel systems. In *Proceedings of the IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems Workshop (PMBS)*, 2018.

[8] A. H. Hunter, Chris Kennelly, Paul Turner, Darryl Gove, Tipp Moseley, and Parthasarathy Ranganathan. Beyond malloc efficiency to fleet efficiency: A hugepage-aware memory allocator. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 257–273, 2021.

[9] Roberto Ierusalimschy, Luiz Henrique de Figueiredo, and Waldemar Celes. Lua—an extensible extension language. *Software: Practice and Experience*, 26(6):635–652, 1996.

[10] Youngjin Kwon, Hangchen Yu, Simon Peter, Christopher J. Rossbach, and Emmett Witchel. Coordinated and efficient huge page management with Ingens. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 705–721, 2016.

[11] Taehyung Lee, Sumit Kumar Monga, Changwoo Min, and Young Ik Eom. MEMTIS: Efficient memory tiering with dynamic page classification and page size determination. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

[12] Huaicheng Li et al. Pond: CXL-based memory pooling systems for cloud platforms. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[13] The Linux Kernel Documentation. *Transparent Hugepage Support*. Documentation/admin-guide/mm/transhuge.rst.

[14] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[15] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. TPP: Transparent page placement for CXL-enabled tiered-memory. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[16] Nafis Mustakin. From source code to sampling correction: Predicting memory-footprint models for unseen programs. Manuscript, 2026.

[17] Nicholas Nethercote and Julian Seward. *DHAT: a dynamic heap analysis tool*. Valgrind user manual, `https://valgrind.org/docs/manual/dh-manual.html`.

[18] Nicholas Nethercote and Julian Seward. Valgrind: a framework for heavyweight dynamic binary instrumentation. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '07, page 89–100, New York, NY, USA, 2007. Association for Computing Machinery.

[19] Aleix Roca Nonell, Balazs Gerofi, Leonardo Bautista-Gomez, Dominique Martinet, Vicenç Beltran Querol, and Yutaka Ishikawa. On the applicability of pebs based online memory access tracking for heterogeneous memory management at scale. In *Proceedings of the Workshop on Memory Centric High Performance Computing*, MCHPC'18, page 50–57, New York, NY, USA, 2018. Association for Computing Machinery.

[20] Ashish Panwar, Sorav Bansal, and K. Gopinath. HawkEye: Efficient fine-grained OS support for huge pages. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.

[21] SeongJae Park, Yunjae Lee, and Heon Y. Yeom. Profiling dynamic data access patterns with controlled overhead and quality. In *Proceedings of the 20th International Middleware Conference Industrial Track*, pages 1–7, 2019.

[22] Ivy Peng et al. On the memory underutilization: Exploring disaggregated memory on HPC systems. In *International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, 2020.

[23] Bobby Powers, David Tench, Emery D. Berger, and Andrew McGregor. Mesh: Compacting memory management for C/C++ applications. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2019.

[24] Amanda Raybuck, Tim Stamler, Wei Zhang, Mattan Erez, and Simon Peter. HeMem: Scalable tiered memory management for big data applications and real NVM. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2021.

[25] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Memory demands in disaggregated HPC: How accurate do we need to be? In *International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2021.

[26] Felippe Vieira Zacarias, Paul Carpenter, and Vinicius Petrucci. Dynamic memory provisioning on disaggregated HPC systems. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC-W)*, 2023.